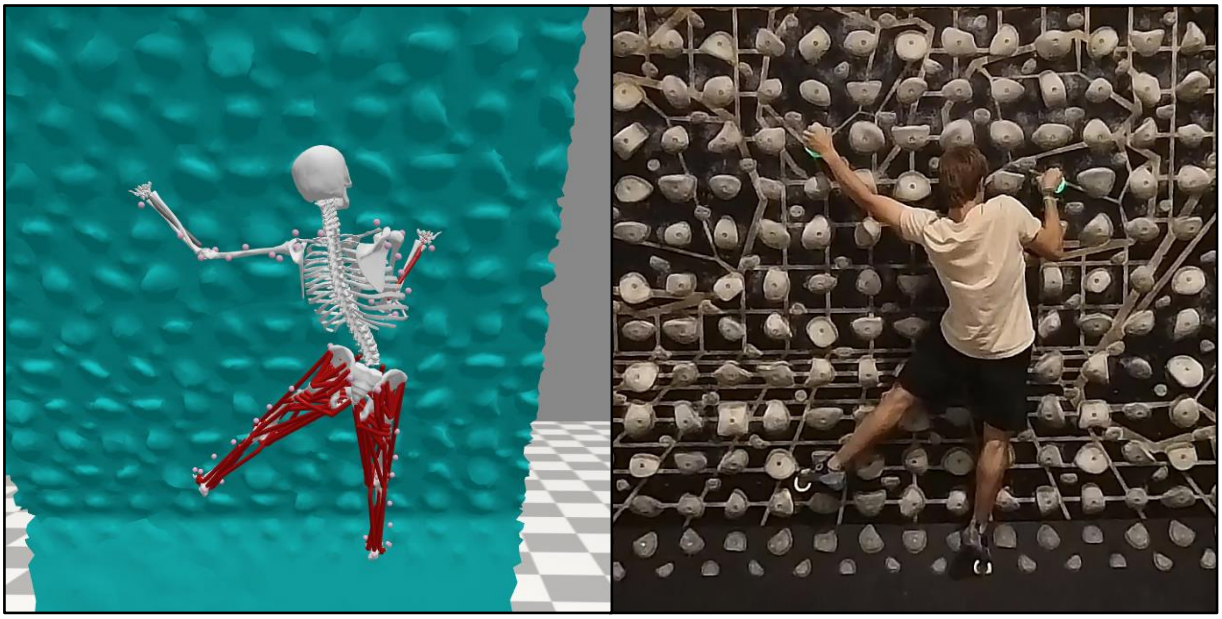
SIMULATIVE KRÄFTEOPTIMIERUNG BEIM BOULDERN

Minimieren der Fingerkraft mit einem iterativen Prozess in einer Computersimulation



Till Fischer

Maturaarbeit

Kantonsschule Wettingen

31.10.2025 Würenlos

Betreuende Lehrperson: Mathilde Rüfenacht
Gegenlesende Lehrperson: Nicole Roth

Zusammenfassung

Diese Arbeit untersucht durch die Entwicklung eines numerischen Programms in Python die Frage, inwiefern Kletterpositionen im Bouldern mit physikalischen Simulationen optimiert werden können. Dafür wurde ein Optimierungsprozess, basierend auf den Methoden Static Optimization und Inverse Kinematics (zur Bestimmung von Körperpositionen) sowie physikalischen Berechnungen für die Kräfte, implementiert. Die Resultate aus diesen Berechnungen wurden anhand theoretischer Aspekte des Boulderns und im Vergleich an einer echten Kletterwand ausgewertet. Der Prozess kam nicht an optimale Positionen, die möglichst wenig Fingerkraft brauchen, heran, aber das Erfüllen mancher Aspekte des Boulderns zeigte, dass eine Weiterentwicklung doch zu einer möglichen Methodik zur Optimierung von Kletterpositionen führen könnte.

Vorwort

Das Thema dieser Arbeit verbindet meine persönlichen Interessen und Hobbys, das Bouldern und die Informatik, mit meinem schulischen Interesse an der Physik. Seit dem Aufschwung von Künstlicher Intelligenz gibt es erste Entwicklungen beim Simulieren von Kletterpositionen beim Bouldern. So wurde ich darauf aufmerksam, dass sich diese Themengebiete verbinden lassen. Allerdings wollte ich nicht auf der Ebene der Künstlichen Intelligenz, sondern lieber an einer eigenen physikalischen Simulation arbeiten, da ich mich fragte, ob ich mein physikalisches Wissen und meine Informatikkenntnisse auf ein solches reales Problem anwenden kann.

Besonderer Dank gilt Mathilde Rüfenacht für das Betreuen dieser Maturaarbeit. Dazu gehören vor allem das Weiterhelfen bei inhaltlichen Themen und Lösungsfindungen, sowie Rückmeldungen zur Arbeit und Inputs zu sehr hilfreichen Literaturquellen. Ausserdem möchte ich meinem Vater für die anregenden Diskussionen zum Inhalt und die Rückmeldungen zur schriftlichen Arbeit danken.

Auch zur Umsetzung dieser Arbeit hat SimTK durch ihre frei zur Verfügung gestellte Software OpenSim beigetragen, die für Simulationen des menschlichen Körpers genutzt wurde und grundlegend für die praktische Umsetzung dieser Arbeit war. Ausserdem geholfen hat die Boulderhalle Bouba (Boulder Baden), an deren Kletterwand die in dieser Arbeit gezeigten Kletterpositionen getestet werden konnten.

Zuletzt wurde ChatGPT zur Unterstützung bei der Problembehebung in den Python Programmen bei der Umsetzung der Arbeit verwendet und hat spezifische Fragen zur Implementierung einzelner Funktionen in OpenSim geklärt, da wenig Literatur dazu vorhanden ist.

Beiträge

Mathilde Rüfenacht

- E-Mail: RueM@kswe.ch
- Betreuung der Arbeit und erklären Inhaltlicher Themen

Beat Fischer

- E-Mail: beatfischer@gmx.ch
- Gegenlesen der Arbeit

Inhaltsverzeichnis

Zusammenfassung I
Vorwort II
Inhaltsverzeichnis III
1 Einleitung 1
2 Theorie 3
2.1 Bouldern 3
2.2 Modell und Optimierungsprozess 4
2.3 Mechanik 6
2.4 Bestimmen der Körperposition 6
2.4.1 Forward Kinematics 6
2.4.2 Inverse Kinematics 8
2.4.3 Übertragung auf den menschlichen Körper in OpenSim 9
2.4.4 Inverse Kinematics in OpenSim 11
2.5 Bestimmung der statischen Kräfte 13
2.5.1 Static Optimization Ansatz 13
2.5.2 Vereinfachungen des Kräftemodells 13
2.5.3 Statische Kräfte Gleichungen 15
2.5.4 Funktion für die Fingerkraft 15
2.6 Gradient Descent 17
2.6.1 Globales Minimum 18
2.7 Visualisierung mit OpenSim 19
2.8 Anwendung der Optimierung an einem Kilter Board 20
3 Resultate 21
3.1 Verlauf des Optimierungsprozesses mit dem Gradient Descent 21
3.2 Vergleich der berechneten Körperpositionen mit realen Situationen 22
3.3 Vergleich mit dem gemessenen Kraftaufwand in realen Situationen 27
4 Diskussion 29
4.1 Vergleich der berechneten Positionen mit der Bouldertheorie 29
4.2 Grenzen und Fehlerquellen bei den Messungen 30
4.3 Grenzen und Fehlerquellen des Optimierungsprozesses 31
4.4 Problematik eines fehlerhaften Gleichungssystems 32

4.4.1 Überarbeitete Variante der Berechnungen der Fingerkraft 33
4.5 Schlussfolgerung 34
Literaturverzeichnis 35
Abbildungsverzeichnis 37
Anhang A 38
Anhang B 48

1 Einleitung

Bouldern ist eine Sportart, die in den letzten Jahren immer beliebter wurde, weil alle sie ohne viel Vorwissen und Material ausprobieren können. Trotzdem ist das Bouldern vielseitig und beansprucht sowohl physische Stärke als auch Technik und Präzision. Beides lässt sich trainieren, wobei es beim technischen Aspekt schwieriger ist, da es nicht einfach eine optimale Technik gibt, die man lernen kann, sondern diese sehr spezifisch für jede Person und Kletterroute ist. Trotzdem gibt es Regeln, die man beachten kann, um das Bouldern einfacher zu machen. Teil des technischen Wissens ist das Routenlesen. Dabei geht es darum, in einer bestehenden Wand einen Weg zu finden, die Route so zu klettern, dass sie physisch möglich ist und nicht zu viel Kraft braucht.

Eine Kletterwand ist statisch und es gibt wenig äussere Einflüsse wie Wetter oder andere Personen. Alle wirkenden Kräfte sind durch die Kletterwand und die kletternde Person vorgegeben. Somit eignet sich die Situation für eine theoretische Simulation und es stellt sich die Frage, ob das Routenlesen, beziehungsweise das Finden einer optimalen Position, nicht auch mit einem Computerprogramm berechnet werden kann. Daraus ergibt sich die Leitfrage dieser Arbeit: Inwiefern können Körperpositionen beim Bouldern mit physikalischen Werten beschrieben und die optimale Position mit einem numerischen (Annäherung einer mathematischen Lösung) Programm berechnet werden?

Das Ziel dieser Arbeit ist, diese Fragestellung mit der Entwicklung eines Programms und dem Vergleich der Resultate mit echten Situationen zu beantworten. Als Input für das Programm werden jeweils zwei Klettergriffe für die Position der Hände und zwei für die Füsse vorgegeben (die Kletterposition). Anhand dieser soll das Programm eine ideale Körperposition (Körperhaltung aller Gelenke und Körperteile) berechnen, die die Kraft, welche auf die Finger wirkt, minimiert, da diese Kraft oft der limitierende Faktor beim längeren Klettern ist. Die Bewegung zwischen den Kletterpositionen wird vorerst nicht betrachtet, da dies für diese Arbeit zu kompliziert wäre. Die Positionen des Kletterers bleiben also statisch. Trotzdem kann das Wissen über eine solche Position bereits helfen, beim Bouldern voranzukommen, ohne dabei zu viel Kraft zu verbrauchen.

Sollte sich das Simulieren als erfolgreich herausstellen, so könnte eine Weiterentwicklung beispielsweise zu einer Lernapp führen, bei der das Routenlesen an fiktiven Routen mit Korrektur durch das Programm geübt werden kann. Weil auch die Entwicklung von Künstlicher Intelligenz beim Bouldern schon fortgeschritten ist (Belay AI, 2025), könnten die theoretisch optimierten Positionen einer Simulation zum Lernen dieser KI verwendet werden, vor allem da sich damit unendlich viele Datensätze generieren lassen.

2 Theorie

2.1 Bouldern

Bouldern ist das Klettern ohne Seil oder Sicherung in Absprunghöhe (Spirkl, 2025). Dies kann an Felsblöcken oder auch im Innenbereich an künstlich nachgebauten Kletterwänden stattfinden. Sowohl koordinative Boulderprobleme (eine Route aus aufeinanderfolgenden Griffen, die vom Boden bis nach ganz oben, zu einem Top führen) als auch steile Überhänge oder dynamische Sprünge fordern physische Stärke, gute Balance und präzise Technik (Spirkl, 2025).

Es gibt einige wichtige technische Grundlagen beim Bouldern, die das Klettern verbessern können. Hier aufgeführt werden die, welche in statischen Positionen oder für die aufgewendete Kraft besonders bedeutsam sind. Eine wichtige Grundlage ist das Strecken der Arme, wodurch die Zugkraft weniger mit den Muskeln und mehr mit dem Skelett ausgeübt wird. Somit kann vor allem die Arm- und Fingerkraft reduziert werden (Flanagan, 2013, S. 20).

Eine weitere wichtige Technik ist, die Hüfte und damit den Schwerpunkt nahe an der Wand zu halten. Dadurch kann das Gewicht mehr auf die Füsse verlagert werden, was wiederum ein Weg ist, die Kraft in den Armen zu verringern. Besonders nützlich ist diese Verlagerung, weil die Beine deutlich mehr Kraft haben als die Arme, was das ausdauernde Klettern erleichtern kann. Deshalb ist auch die Balance ein essenzieller Teil des Kletterns.

Die Balance beschreibt die Position des Körpers relativ zu den Griffen (Flanagan, 2013, S. 22). Die richtige Balance in einer Position zu finden, kann die Verteilung der Kraft auf Arme und Beine verbessern und so den Aufwand in den einzelnen Körperteilen optimieren. Dadurch kann das Klettern auf einmal viel leichter und entspannter wirken (Flanagan, 2013, S. 22).

Diese technischen Aspekte werden in der Simulation dieser Arbeit nicht berücksichtigt. Trotzdem kann verglichen werden, ob ein Ansatz, der rein auf physikalischen Kräften beruht, mit diesem technischen Wissen vereinbar ist. Daher werden die errechneten Idealpositionen der Simulation später anhand dieser Theorie analysiert.

2.2 Modell und Optimierungsprozess

Die Simulation der Körperposition erfolgt mit einer Software namens OpenSim, die ein Modell des menschlichen Körpers in verschiedenen Positionen darstellen und die Kräfte in den einzelnen Muskeln berechnen kann. Die für die Berechnung einer stabilen Kletterposition wichtigen Kräfte sind die Gewichtskraft, die nach unten wirkt, und die vier Kräfte an den Kontaktpositionen der Hände und Füsse an der Wand, die der Gewichtskraft entgegenwirken.

Wenn nun OpenSim für die Vorgabe einer Kletterposition (Position von Händen und Füssen) eine mögliche Körperposition bestimmt, so können anhand der Gewichtskraft die Kräfte in Händen und Füssen berechnet und mit den Kräften einer anderen Position verglichen werden. Die Körperposition wird nun optimiert, sodass die wirkenden Kräfte möglichst klein werden. Dieser Optimierungsprozess läuft in drei sich wiederholenden Schritten ab, das Darstellen der Position, das Berechnen der vier unbekannten Kräfte (nur die Gewichtskraft ist bekannt) und das Anpassen einer Variabel, um den Prozess leicht verändert von neuem zu starten.

Für eine gegebene Kletterposition soll die optimale Körperposition gefunden werden, sodass die Fingerkraft minimiert wird. Das Schwierige daran ist, dass es viele mögliche Positionen für alle Körperteile gibt, weshalb die Abhängigkeit der Fingerkraft von der Körperposition nicht mit einer einfachen mathematischen Berechnung beschrieben werden kann. Deshalb macht es Sinn, eine zusätzliche bekannte räumliche Variable zu den Hand- und Fusspositionen einzuführen, deren Position aber verändert werden kann und anhand dieser die Körperposition optimiert werden kann. Da die Balance ausgehend vom Schwerpunkt beim Bouldern so wichtig ist, eignet sich die Position des Schwerpunktes hervorragend als zusätzliche Variable zum Optimieren der Körperposition. Mit einem bekannten Schwerpunkt wird die Anzahl möglicher Körperpositionen minimiert und es können einfacher passende Körperhaltungen gefunden werden. Zusätzlich erlaubt eine statische Position mit bekanntem Schwerpunkt Berechnungen anhand der Gesetze der Physik, da die Gewichtskraft bekanntlich im Schwerpunkt ansetzt und so für die Bestimmung der vorhandenen Kräfte verwendet werden kann.

Somit ergibt sich der Grundaufbau des Optimierungsprozesses, den das Programm ausführen soll. Mit den Positionen der Griffe und der Gewichtskraft als Input und der

Lage des Schwerpunktes als Annahme wird jeweils eine Körperposition bestimmt, dann die Kräfte auf die Finger berechnet und anhand dieser Berechnungen die Position des Schwerpunktes verbessert (Abbildung 1). Für eine solche schrittweise Verbesserung eines Parameters eignet sich ein sogenannter Gradient Descent als Optimierungsalgorithmus, der das Minimum einer Funktion bestimmt, indem er sich entlang des Funktionsgraphen bewegt. Für die Bestimmung der Körperposition gibt es fertige Algorithmen auf anatomischer Basis (Inverse Kinematics) und auch für die Berechnung der Kräfte können bekannte physikalische Grundlagen gebraucht werden. Die einzelnen Aspekte dieses Grundaufbaus des Prozesses werden nun aufgeführt.

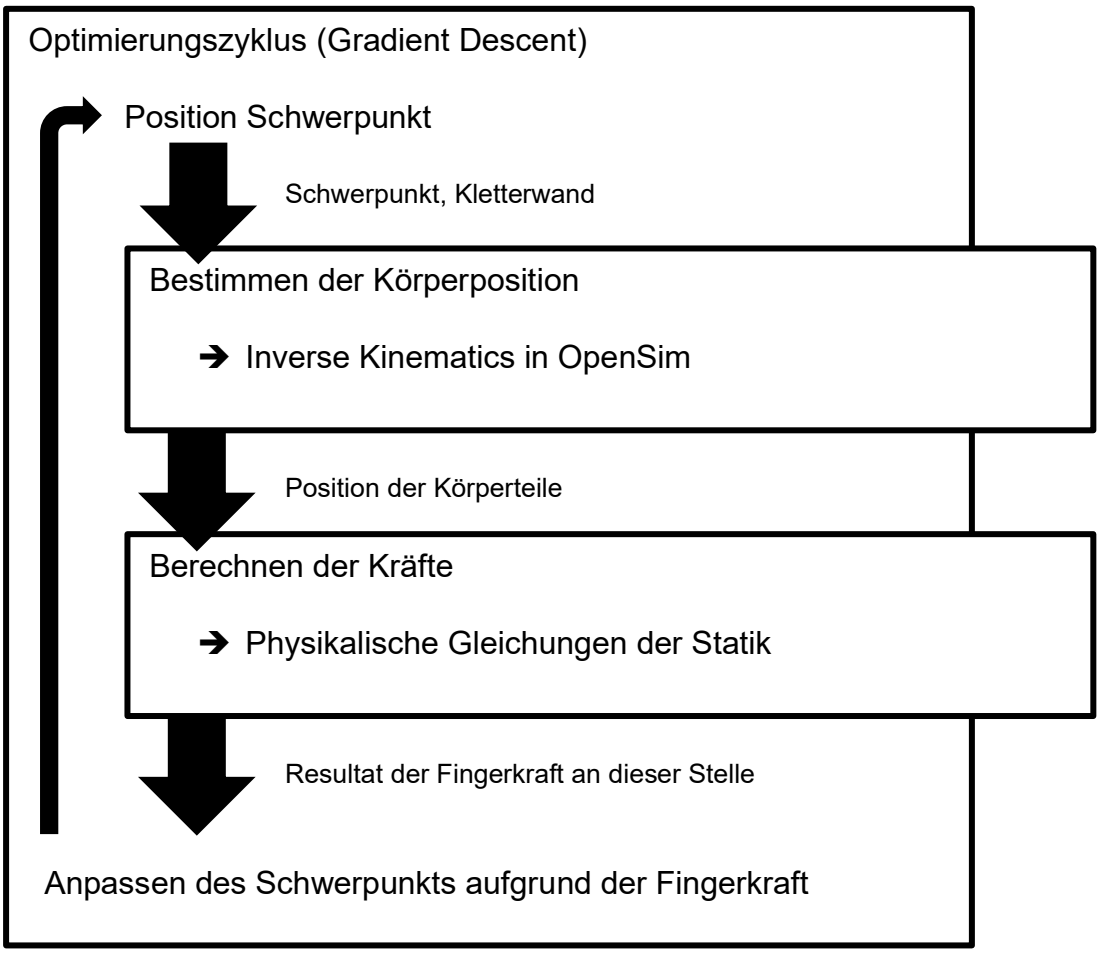


Abbildung 1: Ablauf des Optimierungsprozesses. *Eigene Darstellung*

2.3 Mechanik

Grundlegend für die Simulation einer Kletterposition ist deren Übertragung in die Mechanik. Dabei kann der menschliche Körper als Mechanismus gesehen werden, bei dem starre Teile (physikalische Objekte) durch Gelenke verbunden werden und so miteinander interagieren können (Lynch & Park, 2017, S. 1). In der Mechanik gibt es für solche Mechanismen mathematische Verfahren zur Berechnung von Grössen wie den Winkeln der Gelenke und den Kräften, die auf die starren Teile wirken, basierend auf einer bestimmten Körperposition. Ebenso wichtig wie die Kräfte sind in einem mechanischen System die Drehmomente, welche die Rotation eines Körpers beeinflussen können. Somit wird nicht nur die Geschwindigkeit eines Objekts, die durch Kräfte beschleunigt wird, sondern auch die Drehgeschwindigkeit in Betracht gezogen. Eines dieser Verfahren zur Berechnung bezeichnet man als Forward Kinematics (Lynch & Park, 2017, S. 135).

2.4 Bestimmen der Körperposition

2.4.1 Forward Kinematics

Bei Forward Kinematics werden, anhand bekannter Winkel und Grössen der starren Teile, die Koordinaten von End-Effektoren (Endpunkte einer Aneinanderreihung von Teilen und Gelenken eines Mechanismus) ausgehend von einem bekannten Startpunkt berechnet. Betrachtet man das Beispiel eines Mechanismus im zweidimensionalen Raum mit mehreren durch Gelenke verbundenen Teilen (Abbildung 2), können die Berechnungen mit gewöhnlicher Trigonometrie durchgeführt werden. Für den End-Effektor {4} gelten in diesem Beispiel folgende Gleichungen für die Koordinaten seiner Position (x, y), basierend auf den Längen der starren Teile $(L_1 - L_3)$ und den dazwischenliegenden Winkeln $(\theta_1 - \theta_3)$:

$$x = L_1 cos\theta_1 + L_2 cos(\theta_1 + \theta_2) + L_3 cos(\theta_1 + \theta_2 + \theta_3)$$

$$y = L_1 sin\theta_1 + L_2 sin(\theta_1 + \theta_2) + L_3 sin(\theta_1 + \theta_2 + \theta_3)$$

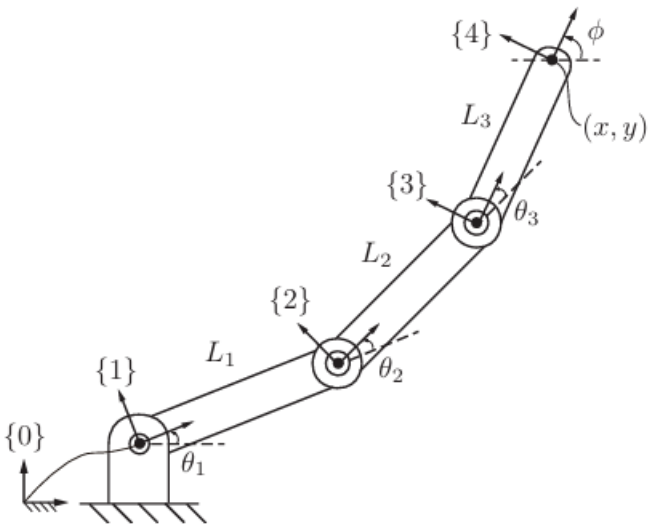


Abbildung 2: Darstellung eines einfachen, zweidimensionalen Mechanismus (L - Länge eines starren Teils, θ - Winkel eines Gelenks, {n} - Index einer Position). *Nach (Lynch & Park, 2017, S. 136).*

Für Mechanismen in dreidimensionalen Räumen reichen diese Berechnungen nicht mehr aus, da es Winkel entlang mehrerer Raumachsen gibt, aber sie können mithilfe von Matrizen allgemein gelöst werden. Dafür wird jeweils die Veränderung der Position durch ein starres Teil mithilfe von zwei Referenzpunkten, die bei benachbarten Gelenken liegen, als Transformationsmatrix dargestellt. Beispielsweise wird die Veränderung der Position zwischen {1} und {2} wie folgt beschrieben:

$$T_{12} = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & L_1 \\ \sin\theta_2 & \cos\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Diese Matrizen können nun auf komplexere Systeme mit mehreren Dimensionen durch Vergrössern angepasst werden. Die Lösung für die Koordinaten der End-Effektoren des gesamten Mechanismus erhält man durch das Multiplizieren aller solcher Transformationsmatrizen vom Ursprung bis zum End-Effektor (Lynch & Park, 2017, S. 136).

So berechnet Forward Kinematics für jeden Mechanismus die Position des End-Effektors basierend auf bekannten Winkeln und Längen der starren Teile. Da in der Problemstellung dieser Arbeit im Gegensatz dazu aber die Positionen der Griffe an der Wand bekannt sind, aber nicht die Körperhaltung, wird ein weiteres, auf Forward Kinematics aufbauendes Verfahren gebraucht, die Inverse Kinematics, um die Körperposition ausgehend von den zu erreichenden Positionen der End-Effektoren (Position der Griffe) zu bestimmen.

2.4.2 Inverse Kinematics

Inverse Kinematics sucht für eine gegebene Position des End-Effektors und bekannte Grössen der einzelnen starren Teile passende Lösungen für die Winkel θ der dazwischenliegenden Gelenke. Dabei kann es im Gegensatz zu Forward Kinematics nicht nur eine, sondern mehrere oder keine mögliche Lösung geben. Ausserdem liefern verschiedene Lösungsansätze unterschiedlich genaue Ergebnisse für die Berechnungen (Lynch & Park, 2017, S. 217).

Bei einfachen Mechanismen mit wenigen Gelenken können die Winkel exakt mit Analytical Inverse Kinematics bestimmt werden (Lynch & Park, 2017, S. 219). Dabei wird das oben beschriebene System aus Gleichungen der Forward Kinematics umgeformt, um die unbekannten Variablen der Winkel zu bestimmen (Lynch & Park, 2017, S. 219).

Für ein komplexeres System, das keine solche analytische Lösung ermöglicht, gibt es die Numerical Inverse Kinematics. Dabei wird das Prinzip angewendet, mit beliebigen Anfangswerten für die Winkel, algorithmisch eine optimale Annäherung der Endposition zu erreichen. Dazu werden die Winkel immer leicht zu neuen Werten aktualisiert, bis die errechnete Lösung für die Position des End-Effektors mittels Forward Kinematics möglichst nahe an den Koordinaten der vorgegebenen Position der Griffe ist. Der zu optimierende Wert ($g(\theta_d)$) ist also die Differenz dieser Lösung für den aktuellen Wert ($f(\theta_d)$) und der bekannten Endposition (x_d): $g(\theta_d) = x_d - f(\theta_d)$. Für die optimale Position muss diese Differenz möglichst null sein (Lynch & Park, 2017, S. 226). Um dies zu erreichen, kann man sich entlang eines Graphen der Differenz ($g(\theta_d)$) abhängig von den aktuellen Winkeln eines Mechanismus bewegen (Abbildung 3). Dafür wird sich entgegengesetzt der Steigung, welche mithilfe der Ableitung ($g'(\theta_d)$)

bestimmt wird, bewegt. Die Ableitung der Forwards Kinematics $f'(\theta_d)$ und so auch die der Differenz kann bestimmt werden, da diese Funktionen, im Gegensatz zum optimalen Wert, bekannt sind. So erfolgt eine Optimierung entlang der Senkung des Graphen (Abbildung 3). Dafür gibt es verschiedene Algorithmen, deren Genauigkeit variiert, auch da sie teilweise auf die Komplexität von Mehrdimensionalität angepasst sein müssen. Beispiele werden in „Modern Robotics: Mechanical, Planning, and Control“ erläutert (Lynch & Park, 2017, S. 227).

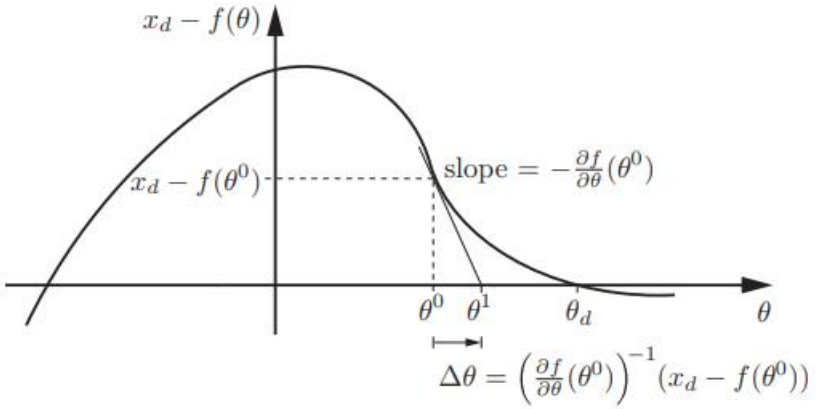


Abbildung 3: Graph einer Optimierung nach numerical Inverse Kinematics (x_d - bekannte Endposition, θ - Winkel der Gelenke eines Mechanismus, θ_d - optimale Winkel, f(θ) - Resultate der Forward Kinematics abhängig von den Winkeln). *Nach (Lynch & Park, 2017, S. 226).*

2.4.3 Übertragung auf den menschlichen Körper in OpenSim

Auch wenn sich der menschliche Körper leicht als Mechanismus definieren lässt, so ist es sehr schwer, alle Komponenten dieses Mechanismus, wie Grösse der starren Teile und Position sowie Freiheitsgrade (um welche Achsen sie sich drehen können) der Gelenke, genau zu definieren. Ein hilfreiches Programm dabei ist OpenSim. OpenSim ist eine Software mit der Möglichkeit zur Simulation von Positionen menschlicher Körper in einem bestimmten Umfeld, zur Visualisierung dieser Positionen, zur Anwendung verschiedener physikalischer Tools (unter anderem Inverse Kinematics) und zur Applikationsentwicklung (API) zum Beispiel mit Python (Hicks, 2024). OpenSim kann also ein Verfahren wie Inverse Kinematics mit wenig Implementationsaufwand in einem Python Programm durchführen. Somit ist es gut

geeignet, um mit akzeptablem Aufwand eine trotzdem weitgehend realistische Simulation des menschlichen Körpers zu erstellen.

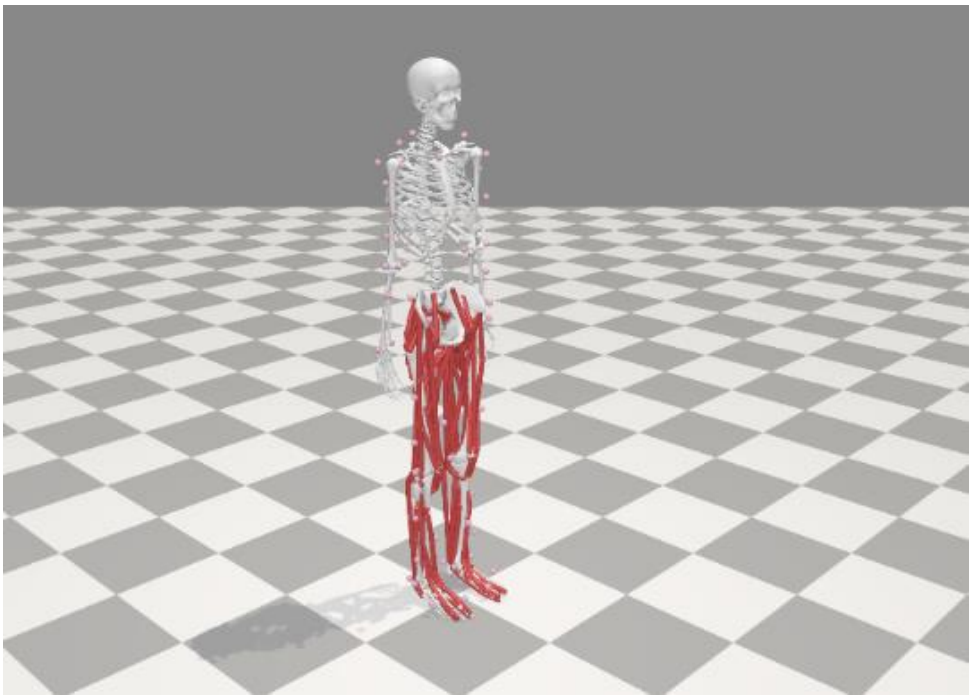


Abbildung 4: OpenSim Modell "FullBodyModel" in OpenSim Benutzeroberfläche. *Eigene Aufnahme.*

Wie der menschliche Körper aufgebaut ist und funktioniert, wird in OpenSim mit vereinfachten Modellen definiert. Darin werden Knochen, Gelenke und Muskeln, aber auch die Umgebung, wie zum Beispiel eine Kletterwand, zusammengefügt und ihre Rahmenbedingungen (Position, Grösse, Bewegungsfreiheit, Kraft) vorgegeben (Hicks, 2020). Solche Modelle sind oft frei zugänglich und werden zum Beispiel für Forschung zur Verfügung gestellt. Für das Simulieren einer Kletterposition eignet sich besonders gut das FullBodyModel von Rajagopal, Apoorva et al. (Rajagopal, 2016) da darin der ganze Körper sowohl mit Knochen und Gelenken als auch mit zusätzlichen Markern (pinke Kugeln im Raum, die relativ zum Körper platziert sind und zum Auslesen der Koordinaten der End-Effektoren verwendet werden können) ausgestattet ist (OpenSim Modell *Rajagopal2015.osim* siehe Github-Repository in Anhang A). Zusätzlich gibt es Muskeln, die als rote Verbindungen dargestellt sind, wobei das verwendete Modell nur Muskeln an den Beinen besitzt (Abbildung 4). Die Muskeln werden während des finalen Optimierungsprozessen ohne Static Optimization nicht verwendet da die Berechnung der Fingerkraft nicht in OpenSim stattfindet, weshalb das Modell auch keine Muskeln am ganzen Körper braucht.

2.4.4 Inverse Kinematics in OpenSim

Das Inverse Kinematics Tool in OpenSim kann das Prinzip der Inverse Kinematics auf ein menschliches Körpermodell anwenden. Der Input der Griffposition bestimmt die gewünschten End-Koordinaten der End-Effektoren. Darauffolgend versucht das Tool alle Gelenkwinkel möglichst so zu bestimmen, dass die Marker der End-Effektoren ihre angestrebte Position erreichen. Dafür wird eine Optimierungsmethode namens Weighted Least Squares (gewichtete kleinste Quadrate) verwendet (Hicks, 2018). Die Methode minimiert die Summe aller quadrierten Differenzen zwischen dem Produkt der vorgegebenen Werte für den Körper (a_i) (Reihe i einer Transformationsmatrix T, mit dem Index i für ein bestimmtes Körperteil) und der zu optimierenden Variabel x (Vektor) und dem eigentlich zu erreichenden Wert für die Position der Marker (b_i) (Vektor): $min\ f(x)\ = \sum_{i=0}^{k} w_i(a_i x - b_i)^2$. Durch die Verwendung von Vektoren für die Variabel x werden alle Gelenke und Dimensionen berücksichtigt. Es gibt noch einen zusätzlichen Wert (w_i), der die Gewichtung einzelner Körperteile bestimmt. Dies macht den «weighted» (gewichteten) Teil der Methode aus. Die Funktion kann auch erweitert werden, zum Beispiel, indem die zwei Summen für Marker und Winkel der Gelenke addiert werden und so die Position und Winkel der Körperteile optimiert werden (Boyd & Vandenberghe, 2004, S. 5). Dies wird auch in OpenSim für Inverse Kinematics gebraucht und sieht wie folgt aus (Hicks, 2018):

$$\min_x \left[\sum_{i \in Marker} w_i(a_i x - b_i)^2 + \sum_{i \in Gelenke} \omega_i(x - c_i)^2 \right]$$

In OpenSim wird nicht nur die angestrebte Position der Marker (b_i) und die der Gelenkwinkel (c_i) berücksichtigt, sondern auch einzelne Gelenkwinkel können als sogenannte „coordinates“ vorgegeben werden. Dadurch sind die Werte dieser bestimmten Winkel schon vorbestimmt und müssen beim Lösen nicht mehr berücksichtigt werden (Hicks, 2018).

Da bei einer optimalen Position der errechnete Wert für $a_i x$ genau die zu erreichende Lösung b_i ist (Differenz = 0), kann ein «Least Square»-Problem analytisch durch lineare Gleichungen gelöst werden. Dabei gilt für das ganze System: $(A)x\ =\ b$, was

sich nach x auflösen lässt. Es gibt viele gute Algorithmen, um solche Probleme zu lösen, die nicht zu zeitintensiv sind (Boyd & Vandenberghe, 2004, S. 4). Welcher in OpenSim genau genutzt wird, ist in der Programmdokumentation nicht spezifiziert, es wird nur erwähnt, dass es sich um einen allgemeinen Ansatz handelt (Hicks, 2018). Eine Implementierung dieses OpenSim Tools in Python ist auch der erste Schritt zur effektiven Umsetzung des Optimierungsprozesses (Python Code *CalculateOpenSimModel.py* siehe Github-Repository in Anhang A).

2.5 Bestimmung der statischen Kräfte

2.5.1 Static Optimization Ansatz

Nachdem die Körperposition mithilfe von Inverse Kinematics in OpenSim bestimmt werden kann, macht es Sinn, zunächst auch für den zweiten Teil, die Berechnung der Kräfte, ein OpenSim Tool zu verwenden. Ein solches ist Static Optimization, welches die auf einzelne Gelenke wirkenden Kräfte in einer gegebenen Position auf die jeweils wirkenden Muskelkräfte aufteilt (Hicks, 2018). Wenn nun die Richtung der Kraft an den Händen und Füssen und die Schwerkraft eines Kletterers vorgegeben werden, so kann dieses Tool zum Beispiel die Kraft, die in jedem Unterarm wirkt, berechnen, vorausgesetzt, dass alle Muskeln im Körper des Modells definiert sind, was bei dem FullBodyModel nicht zutrifft. Daher wurde zuerst ein neues Körpermodell aus verschiedenen Einzelmodellen zusammengebaut, welches am ganzen Körper Muskeln hat (OpenSim Modell *FullBody_SpineBase.osim* siehe Github-Repository in Anhang A). Mit diesem komplexen Modell dauerte die Static Optimization jedoch viel zu lange, da sehr viele Muskeln vorhanden sind, für die die jeweilige Kraft bei der Annäherung der Position jedes Mal bestimmt werden muss. Im Weiteren gab die Implementierung von Static Optimization (Python Code *CalculateOpenSimModel_optimized.py* siehe Github-Repository in Anhang A) oft nur Fehlermeldungen zurück, und falls Werte berechnet wurden, waren diese unrealistisch. Der Grund dafür ist nicht klar, da der genaue Ablauf der Optimierung im Tool sehr kompliziert und undurchschaubar ist. Aufgrund der Schwierigkeiten mit Static Optimization und der hohen Komplexität des Systems, ist es sinnvoll, einen Ansatz zu nutzen, der auf der Basis physikalischer Grundgesetze nur die Grössen der gesuchten Kräfte berechnet und ausserdem eine Vereinfachung für die Richtung der Kräfte benutzt. Dabei wird auch wieder das OpenSim Modell gewechselt und die Optimierung findet wieder mit dem ursprünglichen FullBodyModel statt.

2.5.2 Vereinfachungen des Kräftemodells

Um auf kosteneffiziente Weise auf die Kräfte an den Händen und Füssen zurückzuschliessen, ist eine Vereinfachung des Modells notwendig. Der erste Schritt wurde schon gemacht, indem das Gewicht einzelner Körperteile vernachlässigt und

das Gesamtgewicht als Gewichtskraft in den Schwerpunkt verlegt wurde, eine typische Vereinfachung in der Mechanik. Da neben der Gewichtskraft für eine stabile Position nur die Kräfte an den Kontaktstellen der Hände und der Füsse mit der Kletterwand vorhanden sind, reicht es, das System auf diese fünf Kräfte zu reduzieren. Dabei muss die Summe all dieser Kräfte null sein, damit der Kletterer nicht nach unten fällt oder sich von der Wand abstösst. Da die Gewichtskraft nur in eine Richtung wirkt, nämlich nach unten, sollte das Ziel sein, dass die anderen Kräfte möglichst stark nach oben wirken, weil jede Kraft in eine andere Richtung durch den Kletterer selbst wieder ausgeglichen werden muss und so mehr Kraft bedeutet. Stösst er sich zum Beispiel stark mit den Füssen von der Wand ab, so muss er sich mit den Armen wieder zur Wand hinziehen. Damit also die Kräfte in Händen und Füssen korrekt bestimmt werden können, müssen auch deren Richtungen bekannt sein, um das System eindeutig mit einem Gleichungssystem für statische Systeme lösen zu können. Dafür gibt es die Annahme, dass diese Kräfte eine Erweiterung der Vektoren vom Schwerpunkt zu den Kontaktpunkten an der Wand sind, und so die gleiche Richtung wie diese Vektoren haben (Abbildung 5). Die Richtung der Kräfte wurde in einem Versuch mit Kräftemessung an einer Kletterwand untersucht, was die Vereinfachung bestätigt (Aladdin & Kry, 2012). Daher wurde diese Vereinfachung für die Berechnung der Kräfte in dieser Arbeit genommen.

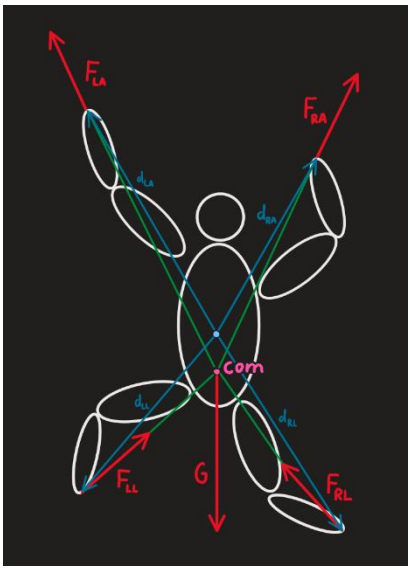


Abbildung 5: Vereinfachung der Kräfte, Modell. *Eigene Darstellung.*

2.5.3 Statische Kräfte Gleichungen

Physikalisch beschreibt eine stabile Kletterposition ein sogenanntes statisches System, bei dem sich sowohl alle Kräfte als auch alle Drehmomente aufheben müssen, wodurch sich folgende Gleichungen für das System ergeben:

$$\vec{F}_{RA} + \vec{F}_{LA} + \vec{F}_{RL} + \vec{F}_{LL} + \vec{G} = 0 \qquad [1]$$

$$\vec{d}_{RA} \times \vec{F}_{RA} + \vec{d}_{LA} \times \vec{F}_{LA} + \vec{d}_{RL} \times \vec{F}_{RL} + \vec{d}_{LL} \times \vec{F}_{LL} + \vec{d}_{MM} \times \vec{G} = 0 \qquad [2]$$

RA, LA, RL, LL sind die Abkürzungen für Right Arm, Left Arm, Right Leg und Left Leg. $\vec{G}$ ist die Gewichtskraft, eine bekannte Grösse. Die Vektoren $\vec{d}$ sind Vektoren von einem frei gewählten Punkt zu den jeweiligen Angriffspunkten der Kräfte an den Griffen. Da sich in einem statischen System die Drehmomente aller Kräfte in jedem beliebigen Punkt aufheben, müssen sie sich auch in diesem frei gewählten Punkt ausgleichen. $\vec{d}_{MM}$ ist der Vektor von diesem Punkt zum Schwerpunkt. Das Drehmoment lässt sich aus dem Kreuzprodukt zwischen dem Vektor $\vec{d}$ und der Kraft $\vec{F}$ bestimmen [2] (Whiteman, 2025). Damit sie sich jeweils aufheben, müssen die Summen aller Drehmomente und aller Kräfte null ergeben [1, 2].

2.5.4 Funktion für die Fingerkraft

Für die Optimierung mit einem Gradient Descent muss aus den Gleichungen eine Funktion aufgestellt werden. In diesem Fall soll sie die Grösse der Kräfte aus den Gleichungen abhängig von der Position des Schwerpunkts (COM) beschreiben. Da das Ziel in dieser Arbeit ist, die Fingerkraft zu minimieren, werden nur die beiden Kräfte in den Armen $\vec{F}_{RA}$ und $\vec{F}_{LA}$ als Resultat der Funktion berechnet, der Einfachheit halber werden sie addiert. Folgende Funktion soll also optimiert werden:

$$F(COM) = |\vec{F}_{RA}| + |\vec{F}_{LA}| \qquad [3]$$

Die Kräfte können, da die Richtung durch die Vereinfachung bekannt ist, zusätzlich wie folgt definiert werden: $\vec{a}$ entspricht jeweils dem Vektor vom Schwerpunkt zum Angriffspunkt an den Griffen und gibt die bekannte Richtung der Kraft an, f entspricht der Magnitude der Kraft.

$$\vec{F}_{RA} = f_{RA} \times \vec{a}_{RA} \quad \vec{F}_{LA} = f_{LA} \times \vec{a}_{LA} \quad \vec{F}_{RL} = f_{RL} \times \vec{a}_{RL} \quad \vec{F}_{LL} = f_{LL} \times \vec{a}_{LL} \qquad [4]$$

Werden nun in der Funktion $F(COM)$ [3] für die beiden Kräfte $\vec{F}_{RA}$ und $\vec{F}_{LA}$ diese Formeln [4] eingesetzt, erhält man eine Funktion abhängig vom Punkt COM, da $\vec{a}$ jeweils von der Position von COM abhängig ist. Die Variable f kann nun durch das Lösen des Gleichungssystems [1] und [2] bestimmt werden, wenn die Kräfte $\vec{F}$ ebenfalls durch die Formeln [4] ersetzt werden (Python Code *solveequations.py* siehe Github-Repository in Anhang A). Durch Einsetzen dieser Lösung f in die Funktion [3] erhalten wir die Funktion zur Optimierung der Fingerkraft. Dafür wird sie jeweils entlang der einzelnen Raumdimensionen partiell abgeleitet, um den Gradienten für den Gradient Descent zu erhalten und damit ein Minimum für die Fingerkraft zu finden (Python Code *CalculateOpenSimModel_derivative.py* siehe Github-Repository in Anhang A).

Das allgemeine Lösungsverfahren für das Gleichungssystem und somit auch für die substituierte Funktion sind zu lang, um sie schriftlich zu lösen. Darum erfolgt dieser Schritt in einem Python Programm. Dafür werden alle Kräfte und Vektoren zuerst in ihre drei Richtungs-Komponenten unterteilt, dann wird damit das Gleichungssystem gelöst und schliesslich die Funktion abgeleitet. Durch späteres Ersetzen der Variablen $\vec{a}, \vec{d}$ und $\vec{G}$ durch die bekannten Grössen aus Gewichtskraft und Körperposition, welche die Position des Schwerpunktes, der Hände und Füsse vorgibt, kann so direkt die numerische Ableitung der Funktion [3] an einer bestimmten Position von COM bestimmt werden, die für den Gradient Descent benutzt wird.

2.6 Gradient Descent

Der Gradient Descent ist ein iteratives Optimierungsverfahren, mit dem das Minimum einer Funktion anhand des Gradienten schrittweise bestimmt werden kann. Der Gradient ist die Gesamtheit der partiellen Ableitungen einer Funktion. Das bedeutet, dass eine Funktion, die von mehreren Variablen abhängig ist, nach jeder dieser Variablen abgeleitet wird, um die einzelnen Ableitungen zu bestimmen. Da die Ableitung der Steigung des Graphen entspricht, kann ein Bruchteil des Wertes der Ableitung bei dieser Position vom aktuellen Wert der Variablen abgezogen werden, um sich in Richtung des Minimums zu bewegen (Abbildung 6). Wie gross dieser Bruchteil ist, wird durch den Faktor α bestimmt, und muss für jede Situation angepasst werden.

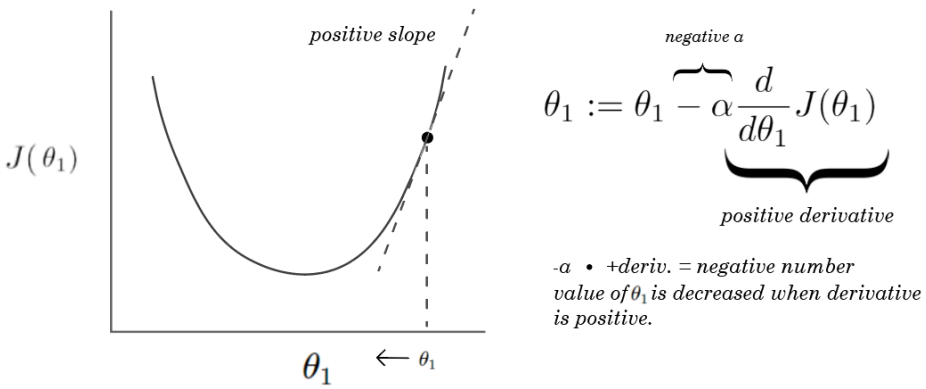


Abbildung 6: Bildliche Darstellung und Formel des Gradient Descent (θ - Variable für die aktuelle Position, J(θ) - Beliebige zu minimierende Funktion, d - Ableitung der Funktion, α - Faktor für die Geschwindigkeit des Gradient Descent). *Nach (Dhalla, 2020).*

Um mit einem Gradient Descent das Minimum für die Fingerkraft zu bestimmen, können zwei Methoden verwendet werden: Mit dem komplizierten Ansatz aus der Static Optimization ist die Optimierung mit einem «angepassten» Gradient Descent einfacher. In dieser Anpassung wird die Steigung des Funktionsgraphen an einem bestimmten Punkt approximiert durch den Quotienten von ΔF (Unterschied der Fingerkraft) und Δx (Veränderung der Position des Schwerpunktes) (Edelman & Johnson, 2023). Diese starke Vereinfachung kann jedoch zu Ungenauigkeiten bei der Lokalisierung lokaler Minima (vor allem in der Situation des Kletterns, bei der es nicht

unbedingt nur ein Minimum gibt) führen, was in der Implementation sichtbar wird (Python Code *GradientDescent.py* siehe Github-Repository in Anhang A).

Sobald die rechnerische Methode und ein Programm zum Ableiten der Funktion vorhanden sind, wird deshalb die zweite Methode, ein «normaler» Gradient Descent in Richtung der Ableitung, möglich und ist deutlich genauer. Dieser Algorithmus wird für den letzten Schritt des Optimierungsprozesses verwendet, indem eine partielle Ableitung entlang drei Dimensionen gemacht wird. Mithilfe der aktuellen Position wird jeweils der Gradient bestimmt und ein Teil seiner Grösse wird der Koordinate abgezogen. So bewegt sich die Koordinate entgegen der Steigung des Graphen, bis der Punkt sich in einem Minimum befindet (Python Code *GradientDescent derivative.py* siehe Github-Repository in Anhang A).

2.6.1 Globales Minimum

In einem komplexen System wie einer Kletterposition gibt es viele verschiedene lokale Minima, an denen der Gradient Descent hängen bleibt. Auch wenn die notwendige Kraft in alle Richtungen zunimmt, ist dies nicht unbedingt die beste Position. Um ein mögliches, global noch besseres Minimum zu finden, kann der Gradient Descent von verschiedenen Startpositionen durchgeführt werden. Deren Resultate werden dann anhand der absoluten Werte für die Kraft sortiert, um ein absolutes Minimum zu finden. Die Startpositionen werden zufällig in einem bestimmten Bereich generiert. Dieser liegt in einem Umkreis von 1 m um den Mittelpunkt der vier Griffpositionen. Die Anzahl der Startpositionen und wie viele Durchläufe des Gradient Descents somit folgen werden, werden ebenfalls durch Ausprobieren bestimmt (Python Code *GradientDescent derivative.py* siehe Github-Repository in Anhang A).

2.7 Visualisierung mit OpenSim

Um die Ergebnisse des Optimierungsprozesses direkt aus dem Programm zu visualisieren, wurde die in der OpenSim API eingebaute Visualisierung verwendet. Dabei kann ein Modell mit einer Bewegung (OpenSim Datei einer Aneinanderreihung von Positionen des Körpers im Modell) dargestellt und diese abgespielt werden. Für jeden Schritt des Gradient Descent, wird die aktuelle Körperposition einer solchen Bewegung als Frame (einzelnes Bild) angefügt und so am Ende des ganzen Prozesses visualisiert. Auch ein 3D Modell der Kletterwand ist hier zu sehen (Abbildung 7). Diese Visualisierung hilft dabei, die Körperpositionen in einer realen Situation an einer Kletterwand auszuprobieren und zu vergleichen (Python Code *CalculateOpenSimModel_derivative.py* siehe Github-Repository in Anhang A).

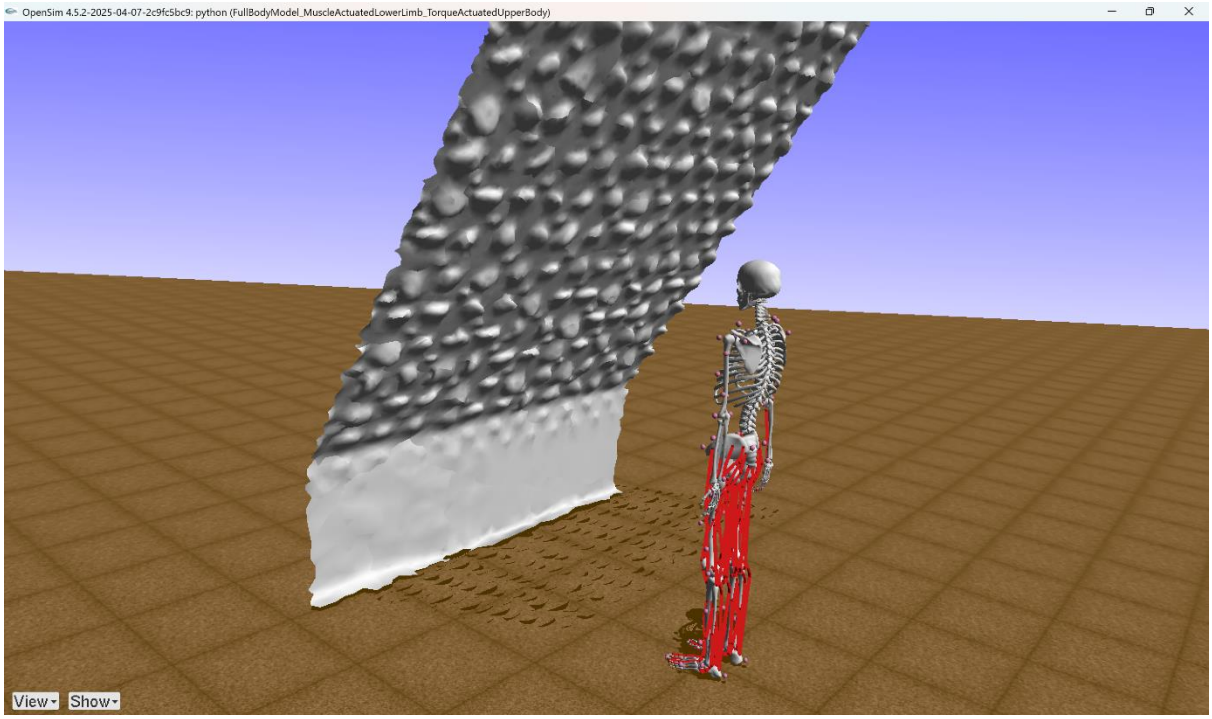


Abbildung 7: Visualisierung des OpenSim Modells mit Kilter Board durch Python. *Eigene Aufnahme.*

2.8 Anwendung der Optimierung an einem Kilter Board

Für den Optimierungsprozess lässt sich die Position von Händen und Füssen definieren. Dadurch wird aber nicht der volle Einfluss von Handpositionierung und Grifftechnik miteinbezogen. Beispielsweise können Griffe je nach Grösse, Art und Orientierung von verschiedenen Seiten oder mit gestreckten oder angezogenen Fingern gegriffen werden. Um deren Einfluss zu minimieren, wurde für die Bestimmung der Griffpositionen eine Route auf einem Kilter Bord, einer kommerziellen Kletterwand, welche in vielen Kletterhallen vorhanden ist und immer gleich aufgebaut ist, gewählt. Das Kilter Board hat sehr viele ähnliche Griffe, die standardisierte Grifftechniken in eine vorgegebene Richtung erfordern. Die für diese Arbeit verwendeten Griffe sind Henkel, die eine Einbuchtung zum Greifen vorweisen, mit einer Grifftiefe zischen ein bis drei Zentimetern (Abbildung 8). Die Griffe wurden mit einer offenen Griff-Technik gegriffen was bedeutet, dass sich die Fingerkuppen oberhalb der restlichen Fingergelenke befinden (Abbildung 9). Die Fusstritte sind klein mit wenig Trittfläche (Abbildung 10) Das Kilterboard lässt sich auch verschieden stark neigen, alle Optimierungen und Messungen wurden bei einer Neigung von 30° durchgeführt. Aus vielen Fotos eines Kilter Boards wurde auf einer Website (Polycam, 2025) ein 3D Modell erstellt, welches für die Visualisierung in OpenSim eingefügt wurde (3D Modell *bouba_kilterboard_cut_red_2.obj* siehe Github-Repository in Anhang A). Die Positionen von Händen und Füssen wurden dann in diesem Modell mit Markern bei ausgewählten Griffen definiert und die Koordinaten dieser Marker im Programm ausgelesen (OpenSim Modell *Rajagopal2015 wall.osim* siehe Github-Repository in Anhang A).

Abbildung 8: Benutzter Klettergriff am Kilterboard. *Eigene Aufnahme.*

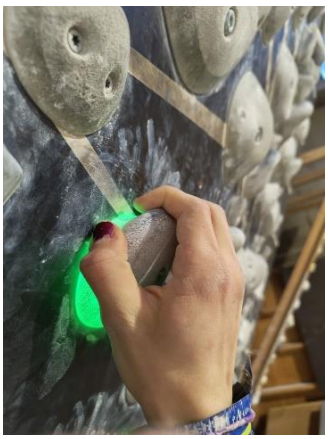


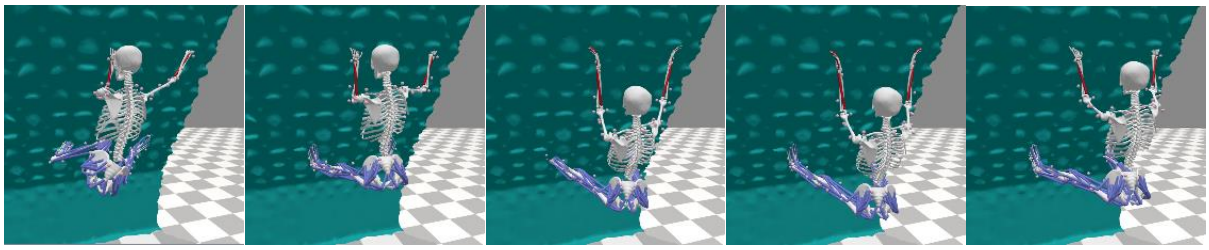
Abbildung 9: Genutzte Grifftechnik (Offener Griff). *Eigene Aufnahme.*

Abbildung 10: Fusstritt am Kilterboard und Tritttechnik. *Eigene Aufnahme.*

3 Resultate

3.1 Verlauf des Optimierungsprozesses mit dem Gradient Descent

Um eine ideale Körperposition für eine bestimmte Kletterposition zu finden, werden die wirkenden Kräfte mit einem Gradient Descent so lange optimiert, bis die aufgewendeten Fingerkräfte möglichst klein sind. Da der Gradient Descent ein schrittweise ablaufender Optimierungsprozess ist, kann man ihn gut visuell anhand der einzelnen Schritte, also den einzelnen berechneten Körperpositionen in jedem Schritt zwischen Start und Optimum, darstellen (Abbildung 11).



Schritt 1 Schritt 3 Schritt 5 Schritt 11 Schritt 18

Abbildung 11: Schritte eines Durchlaufes des Gradient Descent in OpenSim. *Eigene Aufnahme.*

Hier abgebildet sind Ausschnitte eines Durchlaufes des Gradient Descent mit einem einzelnen Startpunkt und 18 Schritten. Der Schwerpunkt verschiebt sich dabei mehr in die Mitte der Klettergriffe, leicht nach unten und weg von der Wand. Die Beine gehen von einer leicht angewinkelten zu einer ausgestreckten Position. Auch die Arme ändern sich zu einer fast ganz gestreckten Haltung. Auffallend ist, dass es zwischen manchen Schritten deutlich grössere Unterschiede als zwischen anderen gibt, bei denen fast gar keine Veränderung zu erkennen ist.

Dies ist ein Beispiel, wie einzelne Durchläufe des Gradient Descent ablaufen können. Allerdings gab es auch viele Situationen, die abgebrochen werden mussten, da die Position zu unrealistisch war (beispielsweise, wenn Körperteile durch die Wand gehen), oder weil die Berechnung fehlschlug, da eine Division durch null stattfand.

3.2 Vergleich der berechneten Körperpositionen mit realen Situationen

Für die Auswertung wurden insgesamt drei Kletterpositionen gewählt, an denen der Optimierungsprozess durchgeführt wurde. Dabei wurde in der finalen Version das FullBodyModel für Inverse Kineamtics in OpenSim und die Visualisierung zusammen mit dem 3D Modell der Kletterwand gebraucht (OpenSim Modell *Rajagopal2015 wall.osim* siehe Github-Repository in Anhang A). Für die Berechnung der Fingerkraft wurde der präsentierte Ansatz mit der Lösung des Gleichungssystems und der Vereinfachung, die die Kräfte als Projektion des Vektors vom Schwerpunkt zu den Griffen definiert, aus Kapitel 2.5 verwendet, beides unabhängig von OpenSim (ohne die Korrektur des Gleichungssystems aus Kapitel 4.4). Die Fingerkraft die so in jedem Schritt berechnet werden kann wird mit dem Gradient Descent möglichst minimiert (Python Code *GradientDescent derivative.py* siehe Github-Repository in Anhang A).

Da für die Simulation bei jeder Kletterposition mehrere Gradient Descent Durchläufe mit verschiedenen Startpositionen getestet werden, kommen auch mehrere Endpositionen als Resultat heraus, mit unterschiedlichen lokalen Minima, an denen der Gradient Descent hängen bleibt. Die Abbildungen 12, 14 und 16 zeigen jeweils die drei besten Körperpositionen aus der Simulation von drei verschiedenen Kletterpositionen mit denselben Bedingungen, aber unterschiedlichen Startpositionen. Ausserdem ist die berechnete Kraft für die beiden Arme angegeben.

Die aus dem Optimierungsprozess entstandenen Körperpositionen wurden zur praktischen Überprüfung an einer realen Kletterwand nachgestellt (Abbildungen 13, 15 und 17). Dafür wurden auf dem Kilter Board die verwendeten Griffe einer bestimmten Position mit den eingebauten LEDs markiert und dann wurde versucht, die als Resultat der Optimierung abgebildete Körperposition in einer realen Situation einzunehmen. Die Streckung der Arme und Beine sowie die Position der Hüfte sollten dabei möglichst mit der Optimierung übereinstimmen, allerdings mussten natürlich auch die anatomischen Grenzen der realen Welt, die nicht alle im Programm miteinbezogen sind, beachtet werden. Wie auch bei der Abbildung des Gradient Descent (Abbildung 11) gab es unrealistische Ausnahmen unter den berechneten Positionen, die anatomisch nicht möglich sind einzunehmen.

Kletterposition 1

1. 1: F = 417 N **1.2**: F = 344 N **1.3**: F = 323 N

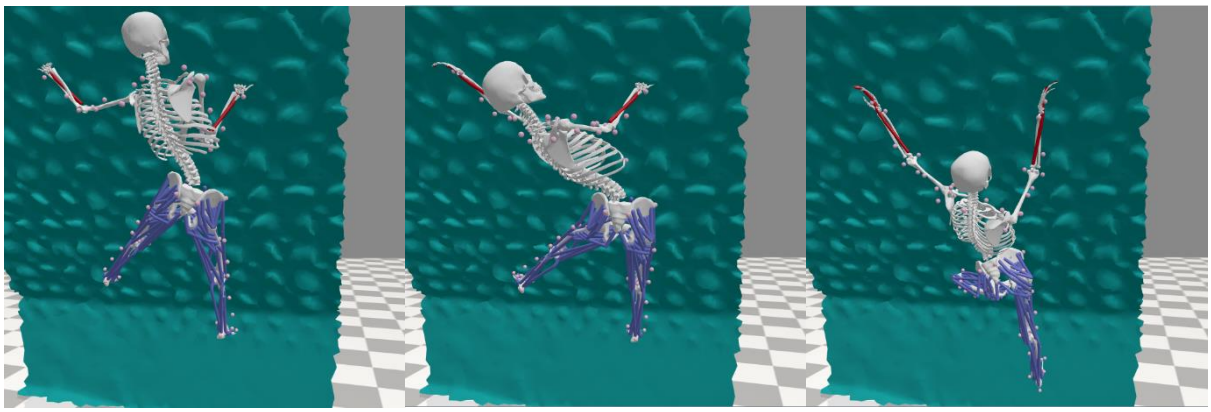


Abbildung 12: Berechnete optimale Körperpositionen für Kletterposition 1 in OpenSim. *Eigene Aufnahme.*

Abbildung 13: Nachstellung der optimalen Körperpositionen 1.1-1.3 am Kilter Board. *Eigene Aufnahme.*

Kletterposition 2

2.1: F = 364 N **2.2**: F = 343 N **2.3**: F = 226 N

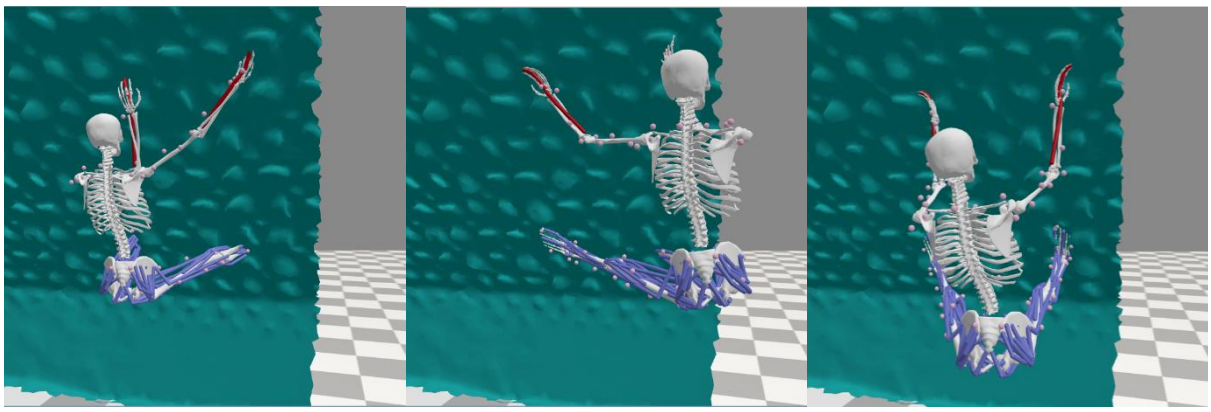


Abbildung 14: Berechnete optimale Körperpositionen für Kletterposition 2 in OpenSim. *Eigene Aufnahme.*

Abbildung 15: Nachstellung der optimalen Körperpositionen 2.1-2.3 am Kilter Board. *Eigene Aufnahme.*

Kletterposition 3

3.1: F = 446 N **3.2**: F = 439 N **3.3**: F = 191 N

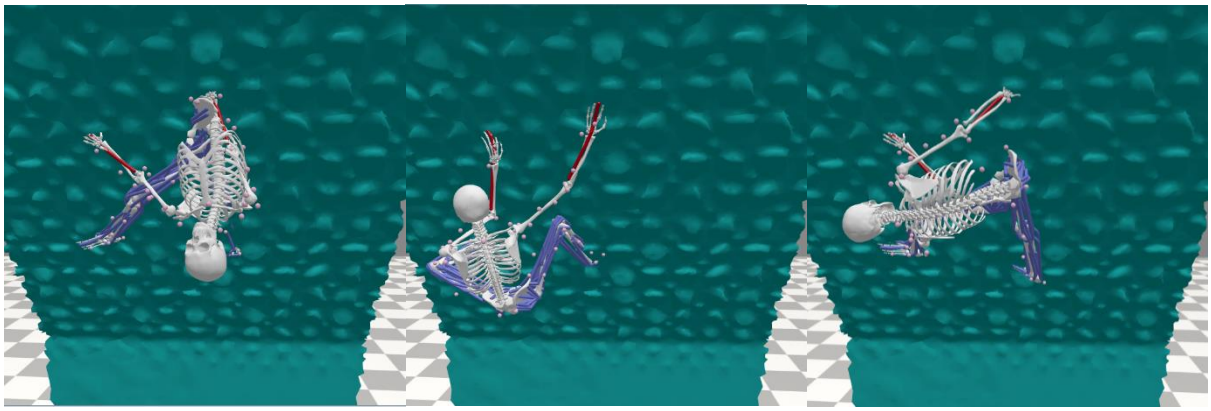


Abbildung 16: Berechnete optimale Körperpositionen für Kletterposition 3 in OpenSim. *Eigene Aufnahme.*

Abbildung 17: Nachstellung der optimalen Körperpositionen 3.1-3.3 am Kilter Board. *Eigene Aufnahme.*

Die Körperpositionen wurden nun anhand der Merkmale des theoretischen Boulderns beschrieben und anschliessend bewertet.

1.1: Diese Körperposition bringt die Hüfte nahe an die Wand, allerdings sind die Beine komplett gestreckt und die Arme angezogen. Der Schwerpunkt befindet sich ungefähr mittig zwischen den Griffen.

1.2: Auch bei dieser Körperposition liegt der Schwerpunkt nahe bei der Wand. Andererseits befindet er sich auf der rechten Seite der Kletterposition. Dies macht die Position sehr einseitig. Die Beine sind immer noch komplett durchgestreckt, genauso wie der linke Arm, der rechte hingegen ist angezogen.

1.3: Die dritte Körperposition hat auch den Schwerpunkt nahe an der Wand. Ausserdem sieht man hier die deutlich mehr gestreckten Arme. Die Beine sind hier in einer unmöglichen Position, das linke Bein geht durch die Wand hindurch, was dazu

führte, dass die reale Umsetzung dieser Position leicht angepasst werden musste. Das Bein wurde angewinkelt, um die Position des Schwerpunktes beizubehalten.

2.1: Die erste Körperposition der zweiten Kletterposition hält den Schwerpunkt weit von der Wand entfernt, auf Höhe der gestreckten Beine. Auch der rechte Arm ist komplett gestreckt, der linke dafür angezogen. Diese Position konnte ebenfalls nicht genauso wie in der Simulation eingenommen werden, da die nah beieinanderliegenden Griffe es nicht ermöglichten, Beine und Arme gestreckt zu haben.

2.2: Ähnlich wie bei der Körperposition 2.1 ist hier der Schwerpunkt weit von der Wand weg, die Arme sind nur leicht angewinkelt. Der Schwerpunkt ist weit nach rechts verschoben. Durch diese Verschiebung nach rechts wird das rechte Bein angewinkelt und das linke gestreckt.

2.3: Bei dieser Körperposition liegt der Schwerpunkt sehr tief und etwa mittig zwischen den Händen. Die Beine sind gestreckt und die Arme nur leicht angewinkelt. Hier zu sehen ist auch, dass die Hände die Griffe nicht ganz berühren, eine Ungenauigkeit, die in einigen Resultaten zu sehen ist.

3.1: Bei Kletterposition drei sind sowohl die erste als auch die dritte Körperposition sehr aussergewöhnlich. Beide konnten in der realen Situation nicht nachgestellt werden, da die Position des Schwerpunktes anatomisch unmöglich ist, bei 3.1 gehen die Arme sogar durch den Körper hindurch. Bei dieser Position ist der Schwerpunkt sehr weit oben, beinahe neben der rechten Hand, aber auch nahe an der Wand. Die Arme sind gestreckt und die Beine zeigen nach unten.

3.2: Dies ist die einzige Körperposition, bei der die Beine deutlich angewinkelt sind und so auch der Schwerpunkt relativ nahe an der Wand ist. Die Arme sind nur leicht angewinkelt. Eine leichte Verschiebung nach links ist vorhanden, wodurch der linke Arm etwas mehr angewinkelt ist.

3.3: Wie auch bei 3.1 ist diese Körperposition so nicht möglich. Ähnliche Merkmale wie bei Position 1 sind vorhanden, wie der wandnahe Schwerpunkt und die nach unten gestreckten Beine, allerdings sind die Arme mehr angewinkelt als bei der ersten Körperposition. Der Schwerpunkt musste in der realen Nachstellung stark verschoben werden.

3.3 Vergleich mit dem gemessenen Kraftaufwand in realen Situationen

Um den berechneten Kraftaufwand der einzelnen Körperpositionen aus den Simulationen einer bestimmten Kletterposition beurteilen zu können, wurde dieser in den real nachgestellten Kletterpositionen bestimmt und verglichen. Dazu wurde so lange in der Körperposition an der Kletterwand gehangen, bis die Position nicht mehr länger gehalten werden konnte und die Muskelkraft versagte. Die maximale Hängezeit wurde gemessen und diente als Basis zum relativen Kraftvergleich und zur Auswertung des Optimierungsprozesses (Abbildung 18). Zwischen den einzelnen Körper- bzw. Kletterpositionen wurden Pausen von mindestens einer Minute eingelegt, um den Einfluss der Erschöpfung zu minimieren. Zusätzlich zu den berechneten Positionen wurde eine freie, aufgrund von Klettererfahrung, gewählte Idealposition gemessen, die als Referenz diente und anhand derer die gemessenen Hängezeiten verglichen wurden. Die Reihenfolge für der einzlnen Körperpositionen innerhalb einer Kletterposition wurde zufällig gewählt. Jede Abfolge dieser Körperpositionen wurde jeweils dreimal wiederholt und für die Resultate der Mittelwert sowie die Standartabweichung bestimmt.

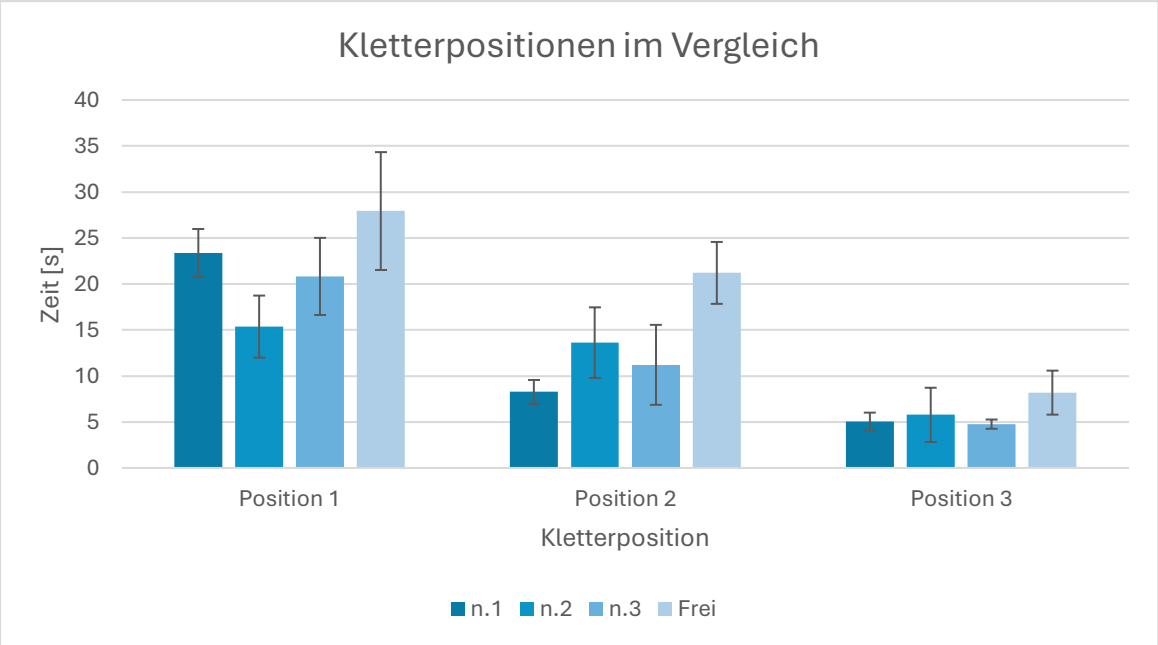


Abbildung 18: Diagramm Kletterpositionen im Vergleich aus Resultaten der Messungen am Kilter Board. *Eigene Darstellung.*

Die maximalen Hängezeiten für die berechneten Positionen der ersten Kletterposition sind alle deutlich unter der frei gewählten Position mit dem kleinsten Abstand bei 1.1 von 4,6 Sekunden. Obwohl nach den Berechnungen aus der Optimierung eigentlich die erste Position (1.1) die schlechteste der drei berechneten Positionen sein sollte, schneidet diese am besten ab. Danach kommt die dritte (1.3), dann die zweite (1.2), was mit den Berechnungen übereinstimmt.

Auch bei der zweiten Kletterposition liegt der höchste Wert bei der freien Position mit dem kleinsten Abstand von 7,6 Sekunden, bei 2.2. Somit erreicht die beste berechnete Position nur knapp zwei Drittel der Hängezeit der freien Position. Hier ist die beste berechnete Position die zweite (2.2), dann die dritte (2.3) und schliesslich, wie berechnet, die erste (2.1).

Bei der dritten Kletterposition sind die berechneten Werte deutlich näher beieinander. Der Unterschied zwischen der besten berechneten Position (3.2) und der schlechtesten (3.3) beträgt nur eine Sekunde oder circa ein Sechstel der Zeit von (3.2). Wiederum ist die freie Position am besten, allerdings erreicht hier die beste berechnete Position (3.2) drei Viertel der Hängezeit der freien Position.

Der kleinste relative Unterschied zwischen einer berechneten und einer freien Position ist innerhalb der ersten Situation zwischen der freien Position und 1.1. Der absolute Wert liegt bei 4,5 Sekunden, was etwas mehr als einem Siebtel der Hängezeit der freien Position entspricht, um die die Körperposition 1.1 kleiner ist.

Freie Körperpositionen

Kletterposition 1 Kletterposition 2 Kletterposition 3

Abbildung 19: Frei Gewählte Körperposition für die Kletterpositionen 1-3. *Eigene Aufnahme.*

4 Diskussion

4.1 Vergleich der berechneten Positionen mit der Bouldertheorie

Bewertet man die Ergebnisse der optimierten Körperpositionen anhand der Theorie zum Bouldern, schneiden die Resultate unterschiedlich gut ab.

In mehreren Positionen (1.1,1.2,1.3, 3.1 & 3.3) ist die Hüfte und somit der Schwerpunkt nahe bei der Wand, was vorteilhaft für die Balance und auch die Fingerkraft ist, da das Gewicht mehr auf die Beine verlagert werden kann und nicht alles in den Armen liegt. Die Beine sind aber nicht in allen diesen Positionen angewinkelt, was am besten wäre, da dadurch wiederum mehr Gewicht auf ihnen liegt, sondern in den meisten Fällen durchgestreckt. Nur eine Position (3.2) hat angewinkelte Beine. Neben den Beinen sind die Arme sehr wichtig. Optimalerweise sind diese möglichst gestreckt, damit die Muskulatur weniger belastet wird und mehr Kraft auf das Skelett wirkt. Dies ist in etwa der Hälfte der Positionen der Fall (1.3, 2.2, 3.1 & 3.2), bei den anderen aber sind die Arme angewinkelt (1.1 & 2.3) oder nur einer der beiden Arme ist gestreckt (1.2, 2.1 & 3.3). Dies ist meistens der Fall, wenn die Position sehr einseitig ist (1.2 & 2.2), also der Schwerpunkt sich stark nach links oder rechts verschoben hat. Dies ist auch schlecht für die Fingerkraft, da die Position schlecht ausbalanciert ist und die daraus resultierende starke Belastung eines Armes schneller zur Erschöpfung dieses Armes führt und so die ganze Position weniger lang gehalten werden kann.

Die Auswertung der verschiedenen Körperpositionen zeigt, dass neben einigen fehlerhaften und unrealistischen Positionen, die es natürlich auch noch gibt, keine Position gefunden wurde, bei der alle Aspekte der Bouldertheorie perfekt sind, sondern immer nur einzelne Regeln erfüllt sind. Es ist jedoch auch kein Muster zu sehen, welche dieser Regeln am besten erfüllt sind. Oft ist eine gute Position des Schwerpunkts zu sehen, was daraus folgen könnte, dass dessen Position direkt beim Optimierungsprozess beteiligt ist.

Auch die Messungen der Hängezeit an der Kletterwand zeigen, dass die berechneten Positionen nicht ganz an die frei gewählte Position, die aus Klettererfahrung und Bouldertheorie entstanden ist, herankommen, auch wenn einige Ergebnisse nahe dran sind. Beispielsweise sind die Positionen 1.1, 2.2 sowie 3.2, die jeweils besten

Resultate der Simulation für jede Kletterposition, nur ein paar Sekunden von der frei gewählten Position entfernt. So zeigt sich, dass sich unter den Resultaten des Optimierungsprozesses doch auch Anhaltspunkte für gute Körperpositionen befinden können und diese nicht komplett zufällig sind. Es gibt aber auch einige Ergebnisse wie Position 2.1 und 1.2, die deutlich schlechter abschneiden als die freie Position, was nochmals bestätigt, dass sich unter den berechneten Positionen einige fehlerhafte Resultate befinden.

4.2 Grenzen und Fehlerquellen bei den Messungen

Bei diesen Messungen gibt es einiges zu berücksichtigen. Die Körperpositionen konnten nur bedingt nachgestellt werden, da einige anatomisch unmöglich waren, und so kann in manchen Fällen das Ergebnis der realen Situation nicht genau mit dem Kraftaufwand für die berechnete Position verglichen werden. Auch spielt die Form der Griffe, die in der Optimierung vernachlässigt wurde, in der realen Situation eine wichtige Rolle. Es kann in manchen Fällen sein, dass für genau diese Griffe eine nicht optimale Position aufgrund der Griffart deutlich besser ist, auch wenn die Hände und Füsse schlussendlich am gleichen Ort sind. Da aber für die freie Position die gleichen Griffe verwendet wurden, ist die Auswirkung der Griffe für den Vergleich weniger schlimm. Trotzdem bleibt sie ein Faktor, der das Resultat beeinflussen kann.

Zuletzt konnten diese realen Messungen nur mit einer Testperson durchgeführt werden, wodurch die statistische Relevanz eher klein ist. Es sind auch nicht alle Aussenfaktoren wie zum Beispiel Erschöpfung komplett eliminiert da alle Messungen zum gleichen Zeitpunkt stattgefunden haben. Deshalb sind die Ergebnisse der Messungen auch nicht die einzige Methode zur Auswertung des Optimierungsprozesses. Sie helfen aber dennoch dabei, die Resultate der Berechnungen mit einer realen Situation zu vergleichen und eine Beurteilung vorzunehmen.

4.3 Grenzen und Fehlerquellen des Optimierungsprozesses

Beim praktischen Überprüfen der Resultate hat sich gezeigt, dass das Nachstellen einiger dieser berechneten Positionen unmöglich ist, da beispielsweise bei gewissen Positionen einzelne Körperteile durch die Wand hindurchgehen (wie bei 1.3). Dies liegt daran, dass der Algorithmus nicht die kompletten Gegebenheiten der Kletterwand kennt und den Körper so nicht perfekt auf deren Struktur anpassen kann. Auch die von der Wand gelösten Hände (2.3) weisen auf ein Problem mit den gegebenen Voraussetzungen hin. Der Algorithmus zum Finden der Körperposition macht einen Optimierungsprozess, der sich einer Lösung annähert, was nicht bedeutet, dass diese immer gefunden wird. So entstehen ab und zu auch Resultate, die von der eigentlichen realistischen Lösung abweichen.

Neben den aufgrund der Kletterwand unmöglichen Positionen, gibt es auch weitere Positionen, die unrealistisch sind (3.1 und 3.2) und es gibt viele Resultate, bei denen gewisse Aspekte boulderthеoretisch schlecht sind. Hier spielen sicherlich auch viele Vereinfachungen eine Rolle: Zum Beispiel sorgt die Reduktion auf den Einfluss von nur fünf Kräften bei der Berechnung der Fingerkraft dafür, dass andere Kräfte, die innerhalb des menschlichen Körpers wirken, nicht mehr betrachtet werden und dieser sich dadurch frei in viele Positionen beugen und drehen kann, auch wenn diese in der Realität zu viel Kraft bräuchten (wie bei 3.1 und 3.2). Ebenfalls wird zur Optimierung der Fingerkraft in der entsprechenden Funktion die Summe der beiden Kräfte in den Armen verwendet, wodurch deren unterschiedliche Belastung nicht beachtet wird, was zu den Positionen mit einer starken Verschiebung auf eine Seite führen kann. Dies könnte mit einer Einbindung der Differenz der beiden Fingerkräfte in die Funktion für die Fingerkraft behoben werden, Die Funktion lautet dann wie folgt: $F(COM) = |\vec{F}_{RA}| + |\vec{F}_{LA}| + \alpha(|\vec{F}_{RA}| - |\vec{F}_{LA}|)$ wobei α durch Ausprobieren bestimmt wird. Eine einfache Änderung, die aber etwas Zeit in Anspruch nimmt, da ein weiterer Wert optimiert werden muss.

Auch wenn die vielen Vereinfachungen und Bedingungen, die der Prozess hat, zu teilweise fehlerhaften Positionen führen, so war es aufgrund dieser Simulationen doch erst möglich, dass überhaupt vergleichbare Resultate entstanden, die die Stärken und Schwächen eines solch vereinfachten Modells aufzeigen.

4.4 Problematik eines fehlerhaften Gleichungssystems

Neben den erwähnten Gründen für mögliche Fehlerquellen, trat während der Berechnung der Kräfte für die Körperposition eine schwerwiegende Problematik auf, die nicht mit der Vereinfachung des Systems erklärt werden kann, nämlich der mehrfache Abbruch des Optimierungsprozesses durch die unerlaubte Division durch null. Es hat sich nun herausgestellt, dass sich dieses Problem mit einer Fehlüberlegung beim Erstellen des Gleichungssystems erklären lässt. Die Problematik lautet wie folgt:

Bei der Vereinfachung des Modells auf fünf Kräfte führen alle fünf Kräfte (Arme, Beine und Gewichtskraft) durch den Schwerpunkt und die Summe dieser Kräfte muss immer null ergeben. Allerdings üben diese Kräfte, wenn sie durch den Schwerpunkt gehen, kein Drehmoment mehr auf diesen aus, und auch in jedem anderen Punkt im System ist das Drehmoment automatisch null, wenn sich alle Kräfte im gleichen Punkt (der Schwerpunkt) aufheben. Somit ist die zweite Gleichung zu den Drehmomenten [2], die für das Gleichungssystem genutzt wird, schon durch die erste Gleichung [1] gegeben und sie sind nach dem Umformen der Gleichungen äquivalent. Die drei Gleichungen aus der Summe der Kräfte bleiben, jedoch fehlt nun eine Gleichung, um alle vier Magnituden der Kräfte f konkret zu bestimmen. Es bleibt ein Freiheitsgrad übrig (die Richtung ist durch die Vereinfachung gegeben, die Magnitude wurde aber durch dieses Gleichungssystem berechnet).

Diese Problematik war nicht von Anfang an offensichtlich, vor allem, da nach dem Lösen des komplizierten Gleichungssystems mit dem Python Programm und dem Einsetzen von Werten doch ein scheinbar mögliches Resultat zurückgegeben wurde, was mit einem solchen unterbestimmten Gleichungssystem nicht möglich sein sollte. Der Grund dafür sind numerische Fehler in der Berechnung, die dann zu einer möglichen, aber rechnerisch völlig falschen Lösung führten. Das Problem ist deshalb sehr spät, erst nach Auswertung der Resultate, offensichtlich geworden. Somit basieren die berechneten optimalen Körperpositionen sowie auch der abgebildete Verlauf des Gradient Descent und somit alle dargestellten und Diskutierten Resultate auf falschen Berechnungen.

4.4.1 Überarbeitete Variante der Berechnungen der Fingerkraft

Eine mögliche Lösung für das Problem des unterbestimmten Gleichungssystems ist die Einführung einer weiteren Bedingung. Normalerweise wäre dies die vierte Gleichung, da diese aber nicht existiert, kann beispielsweise die Bedingung, die Summe aller Kräfte zu minimieren, eingeführt werden. Somit lassen sich die Kräfte und die Ableitung der Funktion doch numerisch bestimmen und können für den Gradient Descent genutzt werden. In Python gibt es für das Minimieren von Variablen in einem Gleichungssystem eine Bibliothek namens cvxpy (CVXPY, 2025) die für eine Implementierung dieser neuen Variante genutzt wurde (Python Code *CalculateOpenSimModel_fingerForceOptimized.py* siehe Github-Repository in Anhang A).

Die einzelnen Durchläufe des Gradient Descent verlaufen mit dem neuen Ansatz tatsächlich reibungsloser, es gibt weniger Sprünge hin und her im Schwerpunkt und so auch weniger Positionen, die abgebrochen werden müssen, da sie unmöglich sind. Allerdings sind die Resultate (Abbildung 20) vergleichsweise nicht besser als die ursprünglich berechneten Körperpositionen. Es sind ähnliche Komponenten wie der nahe Schwerpunkt vorhanden, aber auch seitliche Verschiebungen und angewinkelte Arme sind immer noch Teil der Körperpositionen. Dieser Ansatz ist eine Verbesserung der Optimierung, jedoch lassen sich die Probleme und Grenzen des Prozesses damit noch nicht beheben und liegen weiterhin vor.

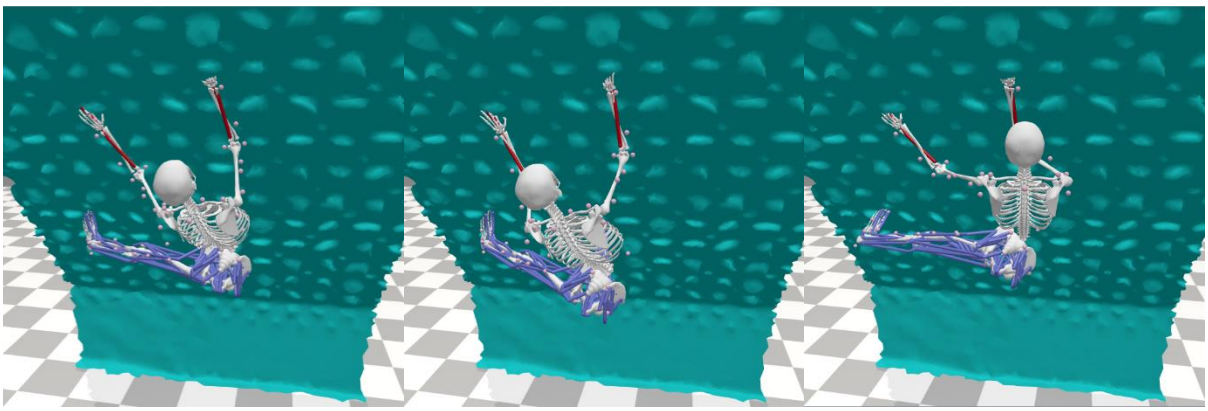


Abbildung 20: Die drei besten Körperpositionen (aufsteigend besser werdend) der überarbeiteten Variante an Kletterposition 3 als vergleich. *Eigene Aufnahme.*

4.5 Schlussfolgerung

Sowohl die theoretische Auswertung als auch die praktischen Messungen weisen darauf hin, dass die Resultate aus den Berechnungen des Optimierungsprozesses nicht an das Optimum beim Bouldern herankommen. Dies lässt sich auf die vielen Vereinfachungen und das Vernachlässigen einiger Faktoren innerhalb des Prozesses zurückführen, ohne die der Prozess aber aufgrund der Komplexität gar nicht erst zustande gekommen wäre. Nicht zu vergessen ist dabei, wie Aspekte, die in der Optimierung stärker berücksichtigt wurden, beispielsweise der Schwerpunkt, doch einen für eine Kletterposition guten Wert erreichen, dies auch in der überarbeiteten Version, in der die Berechnung der Fingerkraft in Bezug auf den Schwerpunkt nicht mehr fehlerhaft ist. Somit kann zumindest das Grundprinzip als erfolgreich bestätigt werden und die Perfektionierung bis hin zur konkreten Anwendung beim Bouldern beruht auf der Weiterentwicklung und Einführung anderer Aspekte, die es beim Klettern zu beachten gilt, in den Optimierungsprozess. Die Fragestellung lässt sich somit wie folgt beantworten: Das grundsätzliche Simulieren physikalischer Werte und das Optimieren der Fingerkraft sind möglich, enden aber im Rahmen dieses Vorgehens manchmal fehlerhaft oder ohne alle Aspekte des Boulderns zu berücksichtigen. Das Prinzip des Simulierens und Optimierens mit einem numerischen Programm kann funktionieren, dieses muss aber dafür aufwändiger aufgebaut und deutlich verfeinert werden.

Das Ziel der Arbeit, ein numerisches Programm zu entwickeln, wurde erreicht und es können damit Rückschlüsse auf die Fragestellung gezogen werden. Das Programm ist ein spezifischer Versuch mit einem spezifischen Ansatz für den Optimierungsprozess, wodurch es nicht stellvertretend für jegliche Art von Optimieren und Simulieren des Boulderns stehen kann. Es ist vielmehr ein Einzelversuch, dessen Erfolg das generelle Prinzip aufzeigt und der zur Weiterentwicklung solcher Programme anregen soll.

Literaturverzeichnis

Aladdin, R., & Kry, P. G. (10. Dezember 2012). *Static Pose Reconstruction with an Instrumented.* Abgerufen am 26.9.25 von McGill: https://www.cs.mcgill.ca/~kry/pubs/vrst12/climbing.pdf

Belay AI. (2025). *AI THAT UNDERSTANDS CLIMBING*. Abgerufen am 24.9.25 von belay.ai: https://belay.ai/

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization.* New York: Cambridge University Press.

CVXPY. (2025). *Welcome to CVXPY 1.7*. Abgerufen am 26.10.25 von cvxpy.org: https://www.cvxpy.org/

Edelman, A., & Johnson, S. (2023). *Fintie-Difference Approximations.* Abgerufen am 26.9.25 von MIT OpenCourseWare: https://ocw.mit.edu/courses/18-s096-matrix-calculus-for-machine-learning-and-beyond-january-iap-2023/mit18_s096iap23_lec04.pdf

Flanagan, D. (2013). *Bouldering for beginners.* Blackrock: Three Rock Books.

Hicks, J. (05. Juli 2018). *How Inverse Kinematics Works*. Abgerufen am 1.9.25 von opensimconfluence: https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53090047/How+Inverse+Kinematics+Works

Hicks, J. (05. Juli 2018). *Static Optimization*. Abgerufen am 31.10.25 von opensimconfluence: https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53090088/Static+Optimization abgerufen

Hicks, J. (29. May 2020). *Model editing*. Abgerufen am 24.9.25 von opensimconfluence: https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53090155/Model+Editing

Hicks, J. (7. Juni 2024). *About OpenSim*. Abgerufen am 24.9.25 von opensimconfluence: https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53114269/About+OpenSim

Lynch, K., & Park, F. (2017). *Modern Robotics: Mechanical, Planning, and Control.* Cambridge: Cambridge University Press.

Polycam. (2025). *Create 3D models from images/video*. Abgerufen am 1.9.25 von Polycam: https://poly.cam/

Rajagopal, A. e. (2016). *Full Body Model for use in Dynamic Simulations of Human Gait*. Abgerufen am 3.6.25 von simtk.org: https://simtk.org/projects/full_body

Spirkl, K. (6.. Juni 2025). *Was ist Bouldern? Alles, was Du über den Klettersport wissen musst*. Abgerufen am 23.9.25 von bergzeit.ch: https://www.bergzeit.ch/magazin/was-ist-bouldern/

Whiteman, W. (2025). *Einführung in die Technische Mechanik*. Abgerufen am 25.9.25 von coursera.org: https://www.coursera.org/learn/engineering-mechanics-statics?

Abbildungsverzeichnis

Abbildung 1: *Eigene Darstellung*

Abbildung 2: Lynch, K., & Park, F. (2017). *Modern Robotics: Mechanical, Planning, and Control.* Cambridge: Cambridge University Press.

Abbildung 3: Lynch, K., & Park, F. (2017). *Modern Robotics: Mechanical, Planning, and Control.* Cambridge: Cambridge University Press.

Abbildung 4: *Eigene Aufnahme.*

Abbildung 5: *Eigene Darstellung.*

Abbildung 6: Dhalla, A. (2020). *The Math of Gradient Descent With Univariate Linear Regression.* Von medium.com: https://medium.com/swlh/the-math-of-machine-learning-i-gradient-descent-with-univariate-linear-regression-2afbfb556131 abgerufen.

Abbildung 7: *Eigene Aufnahme.*

Abbildung 8: *Eigene Aufnahme.*

Abbildung 9: *Eigene Aufnahme.*

Abbildung 10: *Eigene Aufnahme.*

Abbildung 11: *Eigene Aufnahme.*

Abbildung 12: *Eigene Aufnahme.*

Abbildung 13: *Eigene Aufnahme.*

Abbildung 14: *Eigene Aufnahme.*

Abbildung 15: *Eigene Aufnahme.*

Abbildung 16: *Eigene Aufnahme.*

Abbildung 17: *Eigene Aufnahme.*

Abbildung 18: *Eigene Darstellung.*

Abbildung 19: *Eigene Aufnahme.*

Abbildung 20: *Eigene Aufnahme.*

Anhang A

Quellcode und Repository

Der gesamte Code sowie die benutzten OpenSim Modelle und das Modell des Kilter Boards befinden sich im GitHub-Repository:

https://github.com/tillf-web/Maturaarbeit-Till-Fischer

Die wichtigsten Programme sind hier im Anhang nochmals angefügt. Dateinamen:

- CalculateOpenSimModel_derivative.py
- GradienDescent derivative.py

Implementierung der Berechnungen mit dem unterbestimmten Gleichungssystem

```
import opensim as osim
import numpy as np
import os
import xml.etree.ElementTree as ET
import pandas as pd
import sympy as sp
import math

# Configuration
model_path        =        "C:/Users/till/OneDrive        -        Kantonsschule
Wettingen/Schule/MATURARBEIT/ClimbingOptimization/Rajagopal2015 wall.osim"
output_dir        =        "C:/Users/till/OneDrive        -        Kantonsschule
Wettingen/Schule/MATURARBEIT/ClimbingOptimization/output2/"
geometry_path    =    "C:/Users/till/Documents/OpenSim/4.5/Models/FullBodyModel-
4.0/Geometry"
pose_name = "static_climb"
total_mass = 65  # kg
gravity = 9.81
weight = total_mass * gravity
force_each = weight / 4
differneceFactor = 0

# Load and Prepare Model
model = osim.Model(model_path)

# Optional: unlock defaults before initSystem
for i in range(model.getCoordinateSet().getSize()):
    coord = model.getCoordinateSet().get(i)
    coord.setDefaultLocked(False)
```

```
# Initialize
state = model.initSystem()

# Holds you want to extract
hold_names = ["hand_l", "hand_r", "toes_l", "toes_r"]

hold_positions = {}

for comp in model.getComponentsList():
    if isinstance(comp, osim.PhysicalOffsetFrame):
        name = comp.getName()
        if name.endswith("_off"):
            base_name = name[:-4]  # strip "_off"
            if base_name in hold_names:
                location = comp.getPositionInGround(state)
                hold_positions[base_name] = [location.get(i) for i in range(3)]
                  hold_positions[base_name+"_tip"] = [location.get(i) for i in
range(3)]

hold_positions["hand_l_tip"][0]+=0.02
hold_positions["hand_r_tip"][0]+=0.02

# Marker names and corresponding body parts
marker_defs = {
    "RFAradius": hold_positions["hand_r"],  # must match names in model
    "LFAradius": hold_positions["hand_l"],
    "RTOE": hold_positions["toes_r"],
    "LTOE": hold_positions["toes_l"],
}

body_map = {
    "hand_l": "hand_l/hand_l_geom_16",
    "hand_r": "hand_r/hand_r_geom_16",
    "toes_l": "toes_l/toes_l_geom_1",
    "toes_r": "toes_r/toes_r_geom_1",
}

if os.path.exists(os.path.join(output_dir, "summary_pose.mot")):
    os.remove(os.path.join(output_dir, "summary_pose.mot"))

# Unknown forces
f_RA, f_LA, f_RL, f_LL = sp.symbols("f_RA f_LA f_RL f_LL")

# Direction vector components
aRAx, aRAy, aRAz = sp.symbols("aRAx aRAy aRAz")
aLAx, aLAy, aLAz = sp.symbols("aLAx aLAy aLAz")
aRLx, aRLy, aRLz = sp.symbols("aRLx aRLy aRLz")
aLLx, aLLy, aLLz = sp.symbols("aLLx aLLy aLLz")
```

```
# Lever arm components
dRAx, dRAy, dRAz = sp.symbols("dRAx dRAy dRAz")
dLAx, dLAy, dLAz = sp.symbols("dLAx dLAy dLAz")
dRLx, dRLy, dRLz = sp.symbols("dRLx dRLy dRLz")
dLLx, dLLy, dLLz = sp.symbols("dLLx dLLy dLLz")
dMMx, dMMy, dMMz = sp.symbols("dMMx dMMy dMMz")

# Contact Points
RHx, RHy, RHz = sp.symbols("RHx RHy RHz")
LHx, LHy, LHz = sp.symbols("LHx LHy LHz")
RFx, RFy, RFz = sp.symbols("RFx RFy RFz")
LFx, LFy, LFz = sp.symbols("LFx LFy LFz")

# COM
CMx, CMy, CMz = sp.symbols("CMx CMy CMz")

# Gravity components
gx, gy, gz = sp.symbols("gx gy gz")

# Force equilibrium equations
eq1 = sp.Eq(f_RA*aRAx + f_LA*aLAx + f_RL*aRLx + f_LL*aLLx + gx, 0)
eq2 = sp.Eq(f_RA*aRAy + f_LA*aLAy + f_RL*aRLy + f_LL*aLLy + gy, 0)
eq3 = sp.Eq(f_RA*aRAz + f_LA*aLAz + f_RL*aRLz + f_LL*aLLz + gz, 0)

# Moment equilibrium equations (cross products expanded)
eq4 = sp.Eq(f_RA*(dRAy*aRAz - dRAz*aRAy) +
            f_LA*(dLAy*aLAz - dLAz*aLAy) +
            f_RL*(dRLy*aRLz - dRLz*aRLy) +
            f_LL*(dLLy*aLLz - dLLz*aLLy) + (dMMy*gz-dMMz*gy), 0)

F_init = sp.sqrt((f_RA*aRAx) ** 2  + (f_RA*aRAy) ** 2  + (f_RA*aRAz) ** 2)+
sp.sqrt( + (f_LA*aLAx) ** 2 + (f_LA*aLAy) ** 2 + (f_LA*aLAz) ** 2)

F_sub = F_init.subs({
    f_RA: (aLAx*aLLy*aRLy*dLLz*gz - aLAx*aLLy … (shortened force equation)
    f_LA: (aLLx*aRAy*aRLy*dRAz*gz - aLLx*aRAy … (shortened force equation)
})

F = F_sub.subs({
    aRAx: RHx - CMx,
    aRAy: RHy - CMy,
    aRAz: RHz - CMz,
    aLAx: LHx - CMx,
    aLAy: LHy - CMy,
    aLAz: LHz - CMz,
    aRLx: CMx - RFx,
    aRLy: CMy - RFy,
```

```
    aRLz: CMz - RFz,
    aLLx: CMx - LFx,
    aLLy: CMy - LFy,
    aLLz: CMz - LFz,
})

# Partial derivatives
df_dx = sp.diff(F, CMx)
df_dy = sp.diff(F, CMy)
df_dz = sp.diff(F, CMz)


def GetFingerForces(CenterMass, der):
            model_path     =     "C:/Users/till/OneDrive     -     Kantonsschule
Wettingen/Schule/MATURARBEIT/ClimbingOptimization/Rajagopal2015 wall.osim"
    model = osim.Model(model_path)
    for i in range(model.getCoordinateSet().getSize()):
        coord = model.getCoordinateSet().get(i)
        coord.setDefaultLocked(False)

    # Marker names and corresponding body parts
    marker_defs = {
        "RFAradius": hold_positions["hand_r"],  # must match names in model
        "LFAradius": hold_positions["hand_l"],
        "RTOE": hold_positions["toes_r"],
        "LTOE": hold_positions["toes_l"],
        "RFAtip": hold_positions["hand_r_tip"],
        "LFAtip": hold_positions["hand_l_tip"],
        "MASSCENT": CenterMass
    }

    os.makedirs(output_dir, exist_ok=True)

    # Prepare files for Inverse Kinematics
    trc_file = output_dir + pose_name + "_markers.trc"
    # Time settings
    num_frames = 3
    start_time = 0.0
    end_time = 0.02
    time_array = np.linspace(start_time, end_time, num_frames)

    # Marker names and positions (in mm)
    marker_names = list(marker_defs.keys())

    header = [
        f"PathFileType\t4\t(X/Y/Z)\t{trc_file}",
        "DataRate\tCameraRate\tNumFrames\tNumMarkers\tUnits\tOrigDataRate\tOri
gDataStartFrame\tOrigNumFrames",
```

```
        f"100.00\t100.00\t{num_frames}\t{len(marker_names)}\tmm\t100.00\t0\t{n
um_frames}",
        "Frame#\tTime\t" + "\t\t".join(marker_names),
                "\t\t"   +   "\t".join(f"X{i+1}\tY{i+1}\tZ{i+1}"   for   i   in
range(len(marker_names)))
    ]

    rows = []
    for i, t in enumerate(time_array, 1):
        row = [f"{i}", f"{t:.5f}"]
        for name in marker_names:
            x, y, z = [1000 * v for v in marker_defs[name]]  # Convert to mm
            row += [f"{x:.1f}", f"{y:.1f}", f"{z:.1f}"]
        rows.append("\t".join(row))

    with open(trc_file, "w") as f:
        f.write("\n".join(header + rows))

    # Inverse Kinematics
    ik_tool = osim.InverseKinematicsTool()
    ik_tool.setModel(model)
    ik_tool.setName(pose_name + "_IK")
    ik_tool.setMarkerDataFileName(trc_file)
    ik_tool.setStartTime(start_time)
    ik_tool.setEndTime(end_time)
    ik_tool.setOutputMotionFileName(output_dir + pose_name + "_pose.mot")
    ik_tool.set_results_directory(output_dir)
    ik_tool.run()

    model.finalizeConnections()
    state = model.initSystem()
    model.printToXML(output_dir + "model_with_contacts.osim")
    model = osim.Model(output_dir + "model_with_contacts.osim")
    model_path = output_dir + "model_with_contacts.osim"
    motion_file = os.path.join(output_dir, pose_name + "_pose.mot")

    state = model.initSystem()
    # Load motion file (in degrees)
    table = osim.TimeSeriesTable(motion_file)
    labels = table.getColumnLabels()
    row = table.getRowAtIndex(0)

    # Apply the first frame of motion
    for i, label in enumerate(labels):
        if model.getCoordinateSet().contains(label):
            coord = model.getCoordinateSet().get(label)
            value_rad = np.deg2rad(row[i])  # convert from degrees to radians
            coord.setValue(state, value_rad, True)
```

```
model.realizePosition(state)

# Calculate Finger Force

# Get desired marker names
marker_names = [
    "RFAradius", "LFAradius", "RTOE", "LTOE",
    "RShoulder", "LShoulder", "RHip", "LHip", "MASSCENT", "MOMENTCENT"
]

# Read positions of markers
marker_positions = {}

for marker in model.getMarkerSet():
    name = marker.getName()
    if name in marker_names:
        pos = marker.getLocationInGround(state)
        marker_positions[name] = [pos.get(i) for i in range(3)]

# Variable definitions
Values = {
    RHx:marker_positions["RFAradius"][0],
    RHy:marker_positions["RFAradius"][1],
    RHz:marker_positions["RFAradius"][2],
    LHx:marker_positions["LFAradius"][0],
    LHy:marker_positions["LFAradius"][1],
    LHz:marker_positions["LFAradius"][2],
    RFx:marker_positions["RTOE"][0],
    RFy:marker_positions["RTOE"][1],
    RFz:marker_positions["RTOE"][2],
    LFx:marker_positions["LTOE"][0],
    LFy:marker_positions["LTOE"][1],
    LFz:marker_positions["LTOE"][2],
    CMx:marker_positions["MASSCENT"][0],
    CMy:marker_positions["MASSCENT"][1],
    CMz:marker_positions["MASSCENT"][2],
    gx:0,
    gy:-weight,
    gz:0,
   dRAx:marker_positions["RFAradius"][0]-marker_positions["MOMENTCENT"][0]
   dRAy:marker_positions["RFAradius"][1]-marker_positions["MOMENTCENT"][1]
   dRAz:marker_positions["RFAradius"][2]-marker_positions["MOMENTCENT"][2]
   dLAx:marker_positions["LFAradius"][0]-marker_positions["MOMENTCENT"][0]
   dLAy:marker_positions["LFAradius"][1]-marker_positions["MOMENTCENT"][1]
   dLAz:marker_positions["LFAradius"][2]-marker_positions["MOMENTCENT"][2]
    dRLx:marker_positions["RTOE"][0]-marker_positions["MOMENTCENT"][0],
    dRLy:marker_positions["RTOE"][1]-marker_positions["MOMENTCENT"][1],
```

```
        dRLz:marker_positions["RTOE"][2]-marker_positions["MOMENTCENT"][2],
        dLLx:marker_positions["LTOE"][0]-marker_positions["MOMENTCENT"][0],
        dLLy:marker_positions["LTOE"][1]-marker_positions["MOMENTCENT"][1],
        dLLz:marker_positions["LTOE"][2]-marker_positions["MOMENTCENT"][2],
       dMMx:marker_positions["MASSCENT"][0]-marker_positions["MOMENTCENT"][0],
       dMMy:marker_positions["MASSCENT"][1]-marker_positions["MOMENTCENT"][1],
       dMMz:marker_positions["MASSCENT"][2]-marker_positions["MOMENTCENT"][2],
    }

    # Add frame to motion for Visualization

    # Configuration
    source_motion_file = motion_file
    summary_file = os.path.join(output_dir, "summary_pose.mot")

    # Load the original .mot file to get the first frame
    table = osim.TimeSeriesTable(source_motion_file)
    first_row = table.getRowAtIndex(0)
    labels = table.getColumnLabels()

    if not os.path.exists(summary_file):
        # Read and copy the full header from source .mot
        with open(source_motion_file, "r") as fin:
            lines = fin.readlines()
       header_end_index = next(i for i, line in enumerate(lines) if "endheader"
in line.lower()) + 1
        header_lines = lines[:header_end_index]

        # Write header to new file
        with open(summary_file, "w") as fout:
            fout.writelines(header_lines)

        # Create and write one-row table to temp file
        summary_table = osim.TimeSeriesTable()
        summary_table.setColumnLabels(labels)
        summary_table.appendRow(0.0, first_row)
        temp_path = summary_file + ".tmp"
        osim.STOFileAdapter.write(summary_table, temp_path)

        # Append data (excluding auto header) to final file
        with open(temp_path, "r") as ftemp:
            temp_lines = ftemp.readlines()
       data_start = next(i for i, line in enumerate(temp_lines) if "endheader"
in line.lower()) + 1
        with open(summary_file, "a") as fout:
            fout.writelines(temp_lines[data_start:])
        os.remove(temp_path)
        print("Created new summary file with original header.")
```

```
    else:
        # Load existing summary and append first row
        summary_table = osim.TimeSeriesTable(summary_file)
        last_time = summary_table.getIndependentColumn()[-1]
        summary_table.appendRow(last_time + 0.01, first_row)
        osim.STOFileAdapter.write(summary_table, summary_file)
        print("Appended first frame to existing summary.")

    # Return derrivative or funciton
    if der:
        return({df_dx.subs(Values),df_dy.subs(Values),df_dz.subs(Values)})
    else:
        return(F.subs(Values))


def Visualize():
    print("Playing motion in OpenSim Visualizer...")


    # Optional: Add geometry path for visualization
    osim.ModelVisualizer.addDirToGeometrySearchPaths(geometry_path)

    # Disable all muscles
    muscles = model.getMuscles()
    for i in range(muscles.getSize()):
        muscle = muscles.get(i)
        muscle.set_appliesForce(False)

    # Finalize and load motion
    model.finalizeConnections()
    motion = osim.TimeSeriesTable(os.path.join(output_dir, "summary_pose.mot"))

    # Visualize the model with motion
    try:
        osim.VisualizerUtilities.showMotion(model, motion)
    except RuntimeError as e:
        if "VisualizerProtocol" in str(e):
            print("Visualizer closed by user. No problem.")
        else:
            raise  # Re-raise if it's a real problem

    # Reenable all muscles
    muscles = model.getMuscles()
    for i in range(muscles.getSize()):
        muscle = muscles.get(i)
        muscle.set_appliesForce(True)
```

Implementierung des Gradient Descent

```
from CalculateOpenSimModel_derivative import *

# Find middle of all holds
CentX=(hold_positions["hand_r"][0]+hold_positions["hand_l"][0]+hold_positions[
"toes_r"][0]+hold_positions["toes_l"][0])/4
CentY=(hold_positions["hand_r"][1]+hold_positions["hand_l"][1]+hold_positions[
"toes_r"][1]+hold_positions["toes_l"][1])/4
CentZ=(hold_positions["hand_r"][2]+hold_positions["hand_l"][2]+hold_positions[
"toes_r"][2]+hold_positions["toes_l"][2])/4
center_point = (CentX,CentY,CentZ)

# Generate starting position for gradient descent
def random_point_in_sphere(center=(0, 0, 0), radius=1):
    vec = np.random.normal(size=3)
    vec /= np.linalg.norm(vec)
    r = radius * np.cbrt(np.random.rand())
    point = np.array(center) + r * vec
    return tuple(point)

# Factor for gradient descent
alpha=0.00000000000000000001

# Run through Gradient descent (50 times)
s=[]
for _ in range(50):
    P=random_point_in_sphere(center_point)
    x=P[0]
    y=P[1]
    z=P[2]
    # Skip positions with COM outside bounds
    if(y>1.53*(1.4-x)+0.7 or x>1.4):
        continue
    lastJ=[0,0,0]
    # Single steps of gradient descent
    for i in range(20):
        # Calculate body position and finger force
        J=GetFingerForces([x,y,z],1)
        if(len(list(J))<3):
            x+=alpha*list(lastJ)[0]
            y+=alpha*list(lastJ)[1]
            z+=alpha*list(lastJ)[2]
            break
        # Change COM
        x-=alpha*list(J)[0]
        y-=alpha*list(J)[1]
        z-=alpha*list(J)[2]
        # Skip positions with COM outside bounds
```

```
        if(y>1.53*(1.4-x)+0.7 or x>1.4):
            x+=alpha*list(J)[0]
            y+=alpha*list(J)[1]
            z+=alpha*list(J)[2]
            break
        lastJ=J
    V=GetFingerForces([x,y,z],0)
    if(lastJ==[0,0,0]):
        continue
    s.append((V,[x,y,z]))

# Step through all end results again for visualization
s.sort(reverse=True)
for i in s:
    print(i[0])
    GetFingerForces(i[1], 1)

Visualize()
```

Anhang B

Rohdaten der Messungen

Zeit in jeder Gemessenen Körperposition pro durchlauf der Messungen in Sekunden.

Kletterposition 1				
Durchlauf	1.1	1.2	1.3	frei
1	25.59	10.85	23.81	36.07
2	24.81	18.96	23.76	27.30
3	19.72	16.28	14.89	20.41

Kletterposition 2				
Durchlauf	2.1	2.2	2.3	frei
1	7.88	19.05	16.80	25.72
2	10.01	10.92	10.61	20.24
3	6.91	10.90	6.21	17.65

Kletterposition 3				
Durchlauf	3.1	3.2	3.3	frei
1	3.77	9.35	4.88	11.3
2	6.13	5.83	5.31	7.78
3	5.22	2.13	4.09	5.48